Mastering Hibernate ORM & Spring Data JPA -- Internals, Architecture, and Enterprise Usage Guide

---

Author: Senior Java Architect **Target Audience:** Java developers with 3+ years of experience aiming for senior/architect-level roles **Prerequisites:** Core Java, Spring Boot, SQL fundamentals, JDBC basics

---

Table of Contents

1. Introduction to ORM
2. What is Hibernate
3. Hibernate Core Architecture
4. Entity Mapping
5. Hibernate Entity Lifecycle
6. Associations in Hibernate
7. Fetching Strategies
8. Hibernate Query Methods
9. Transaction Management
10. Spring Data JPA Introduction
11. Spring Data JPA Architecture
12. Spring Data JPA Repositories
13. Query Methods in Spring Data JPA
14. Custom Queries
15. Pagination and Sorting
16. Spring Data JPA Specifications
17. Performance Optimization
18. Hibernate Caching
19. Common Performance Problems
20. Real Production Scenario
21. Microservices Data Strategy
22. Design Patterns Used
23. Best Practices
24. Interview Questions (100+)
25. Coding Exercises (20)

Part 1: Introduction to ORM

1.1 What is ORM (Object Relational Mapping)?

ORM is a programming technique that maps objects in an object-oriented language (Java) to rows in a relational database. Instead of writing raw SQL and manually mapping ResultSets to Java objects, ORM frameworks handle this automatically.

Deep Theory: Why ORM Was Invented

The Core Problem: In the early 2000s, enterprise Java projects spent 30-40% of development time writing boilerplate JDBC code. Every entity required manual SQL strings, ResultSet-to-Object mapping, connection lifecycle management, and transaction handling. A single missed `close()` call on a Connection could leak database connections and crash the production server within hours.

What ORM does internally:

1. **Metadata scanning** -- At startup, Hibernate reads all `@Entity` annotations, builds an internal "metamodel" mapping each Java class to its database table, each field to its column
2. **SQL generation** -- When you call `persist(user)`, Hibernate consults its metamodel and the configured Dialect to generate the correct INSERT SQL for your specific database
3. **State tracking** -- Hibernate takes a "snapshot" of every loaded entity. On `flush()` / `commit()`, it compares current field values against the snapshot and generates UPDATE SQL only for changed columns
4. **Connection management** -- Hibernate obtains connections from HikariCP, executes SQL, and returns connections to the pool automatically

```
Without ORM (Raw JDBC):
Java Object  -->  Manual SQL  -->  ResultSet  -->  Manual Mapping  -->  Java
Object

With ORM (Hibernate):
Java Object  <-->  ORM Framework  <-->  Database
              (automatic mapping)
```

Real-World Impact: Why This Matters in Production

Scenario	Without ORM (JDBC)	With ORM (Hibernate)

E-commerce: Add a new field	Change SQL in 15+ DAO methods	Add one field + annotation
Banking: Switch Oracle to PostgreSQL	Rewrite all SQL queries	Change one Dialect property
Healthcare: Audit compliance	Manually track every field change	`@EntityListeners` + Envers
Microservices: 20 entities	20 DAO classes x 5 methods = 100+ methods	20 interfaces, zero implementations

Manual JDBC Example (What ORM Eliminates)

```java
// WITHOUT ORM -- Manual, error-prone, tedious
public User findById(Long id) {
    Connection conn = dataSource.getConnection();
    PreparedStatement ps = conn.prepareStatement(
        "SELECT id, name, email, created_at FROM users WHERE id = ?");
    ps.setLong(1, id);
    ResultSet rs = ps.executeQuery();

    User user = null;
    if (rs.next()) {
        user = new User();
        user.setId(rs.getLong("id"));
        user.setName(rs.getString("name"));
        user.setEmail(rs.getString("email"));
        user.setCreatedAt(rs.getTimestamp("created_at").toLocalDateTime());
    }
    rs.close(); ps.close(); conn.close();
    return user;
}

// WITH ORM (Hibernate/JPA) -- Clean, type-safe, automatic
public User findById(Long id) {
    return entityManager.find(User.class, id); // That's it!
}
```

Production Bug That ORM Prevents

Connection Leak Scenario (JDBC):

```java
// BUG: If ps.executeQuery() throws, conn is NEVER closed
public User findById(Long id) {
    Connection conn = dataSource.getConnection();
```

```
    PreparedStatement ps = conn.prepareStatement("SELECT ...");
    ResultSet rs = ps.executeQuery(); // Exception here -> conn leaked!
    // ... mapping code
    conn.close(); // Never reached
}
// After 20 connections leak, HikariCP pool exhausted -> entire app hangs!
```

With Hibernate, connection lifecycle is managed entirely by the framework. You never open, close, or leak connections.

When NOT to Use ORM

Scenario	Use Instead
Complex reporting with window functions, CTEs	Native SQL or database views
Bulk loading millions of rows	JDBC batch or database COPY command
Simple CRUD with 2-3 tables	Spring Data JDBC (lighter weight)
NoSQL databases (MongoDB, Cassandra)	Spring Data MongoDB/Cassandra

1.2 Object Model vs Relational Model

Deep Theory: The Two Worlds

The object model is designed for **behavior** (methods, encapsulation, polymorphism). The relational model is designed for **data integrity** (normalization, constraints, set-based operations). ORM bridges these fundamentally different paradigms.

Aspect	Object Model (Java)	Relational Model (SQL)
Structure	Classes with fields and methods	Tables with columns
Identity	Object reference (`==`, `.equals()`)	Primary key
Relationships	References (composition, inheritance)	Foreign keys, join tables
Inheritance	Class hierarchy (extends)	No native support
Encapsulation	Private fields + methods	All columns exposed
Navigation	`user.getOrders().get(0).getItems()`	JOIN queries
Collections	`List<Order>`, `Set<Item>`	Separate tables

How Senior Architects Think About This

In Java: You think in terms of object graphs -- a `Customer` HAS a `List<Order>`, each `Order` HAS a `List<OrderItem>`, each `OrderItem` REFERENCES a `Product`. Navigation is natural: `customer.getOrders().get(0).getItems().get(0).getProduct().getName()`.

In SQL: There is no "navigation." To get the same data, you write a 4-table JOIN query. The database has no concept of an "object graph" -- it only knows about rows, columns, and set-based operations.

Hibernate's job is to translate between these two mental models transparently, so developers think in objects while the database stores in tables.

1.3 Object-Relational Impedance Mismatch

The fundamental conflict between how data is represented in objects vs tables:

Mismatch Area	Problem	ORM Solution
Granularity	Object can have fine-grained types (Address, Money)	`@Embeddable` maps value objects
Inheritance	Java has class hierarchies; SQL has no inheritance	`@Inheritance` strategies (SINGLE_TABLE, JOINED, TABLE_PER_CLASS)
Identity	Java: reference equality vs `.equals()`	`@Id` maps to primary key
Associations	Java: object references; SQL: foreign keys	`@ManyToOne`, `@OneToMany` map relationships
Data Navigation	Java: traverse object graph; SQL: explicit JOINs	Lazy loading, fetch strategies

Deep Dive: Why Impedance Mismatch Causes Production Bugs

Granularity Mismatch Example: In Java, a `Money` value object has amount + currency. In SQL, this is just two columns. If you use raw JDBC, you must manually combine/split these everywhere. With `@Embeddable`, Hibernate handles this seamlessly.

Inheritance Mismatch Example: In Java, `CreditCardPayment extends Payment` is natural. In SQL, there's NO inheritance. You must choose: one table with NULL columns (SINGLE_TABLE), separate tables with JOINs (JOINED), or completely separate tables (TABLE_PER_CLASS). Each has performance tradeoffs that senior developers must understand.

Navigation Mismatch Example: In Java, `order.getCustomer().getAddress().getCity()` traverses the object graph, potentially triggering 3 separate SQL queries (LazyInitializationException risk). In SQL, you'd write a single JOIN query. This mismatch is the root cause of the N+1 problem.

1.4 Problems ORM Solves

Data Mapping

```
// ORM maps this automatically:
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;                    // -> id BIGINT PRIMARY KEY

    @Column(nullable = false)
    private String name;                // -> name VARCHAR(255) NOT NULL

    @Column(unique = true)
    private String email;               // -> email VARCHAR(255) UNIQUE

    @Enumerated(EnumType.STRING)
    private UserStatus status;          // -> status VARCHAR(255)

    private LocalDateTime createdAt;    // -> created_at TIMESTAMP
}
```

Identity Management

```
// ORM guarantees: same DB row = same Java object within a session
User u1 = entityManager.find(User.class, 1L);
User u2 = entityManager.find(User.class, 1L);
assert u1 == u2; // TRUE -- same object reference (1st level cache)
```

Why This Matters in Production: Without identity guarantee, you might load the same user twice, modify one copy, and lose changes from the other copy. Hibernate's first-level cache prevents this data inconsistency.

Transaction Handling

```java
// ORM + Spring manages transactions declaratively
@Transactional
public void transferFunds(Long fromId, Long toId, BigDecimal amount) {
    Account from = accountRepository.findById(fromId).orElseThrow();
    Account to = accountRepository.findById(toId).orElseThrow();
    from.debit(amount);
    to.credit(amount);
    // Auto-committed on success, auto-rolled back on exception
}
```

Dirty Checking (Automatic Change Detection)

```java
@Transactional
public void updateUserEmail(Long userId, String newEmail) {
    User user = userRepository.findById(userId).orElseThrow();
    user.setEmail(newEmail);
    // NO save() call needed! Hibernate detects the change automatically.
    // At commit time, Hibernate compares current state vs loaded snapshot
    // and generates: UPDATE users SET email = ? WHERE id = ?
}
```

How Dirty Checking Works Internally:

```
1. findById(1L) -> Hibernate loads User from DB, stores a COPY (snapshot) in
Persistence Context
2. user.setEmail("new@email.com") -> Only the live object changes, snapshot
untouched
3. Transaction commit triggers flush()
4. Hibernate compares: live.email ("new@email.com") vs snapshot.email
("old@email.com")
5. Difference detected -> generates UPDATE SQL for ONLY the changed field
6. SQL executed, snapshot updated
```

Interview Questions for Part 1

Q: What is the object-relational impedance mismatch? A: It's the fundamental conflict between Java's object model (inheritance, encapsulation, object graph navigation) and SQL's relational model (tables,

foreign keys, set-based operations). ORM frameworks like Hibernate bridge this gap by automatically translating between the two paradigms.

Q: When would you choose raw JDBC over Hibernate? A: For bulk data loading (millions of rows), complex analytics with window functions/CTEs, or when absolute query control is needed for performance-critical paths. In practice, most enterprise apps use Hibernate for 90% of operations and native queries for the remaining 10%.

Part 2: What is Hibernate

2.1 What is Hibernate?

Hibernate is the most popular ORM framework for Java. It implements the **JPA (Java Persistence API)** specification and provides additional features beyond the standard.

Aspect	Details
Type	ORM framework
Implements	JPA 3.1 (Jakarta Persistence)
Created by	Gavin King (2001)
Current version	Hibernate 6.x
License	LGPL (open source)

Deep Theory: JPA vs Hibernate -- The Critical Distinction

JPA is an interface (specification). It defines annotations (`@Entity` , `@Id` , `@ManyToOne`) and APIs (`EntityManager` , `Query`) but contains ZERO implementation code. Think of JPA as a Java interface.

Hibernate is the implementation. When you call `entityManager.persist(user)` , the actual code that generates SQL, manages the cache, and talks to JDBC is Hibernate's code.

Why this matters in production:

- Your code should depend on JPA interfaces (`EntityManager` , `@Entity`), NOT Hibernate-specific classes
- This allows you to theoretically swap Hibernate for EclipseLink or OpenJPA (though this rarely happens in practice)
- Spring Boot auto-configures Hibernate as the default JPA provider

```
Your Code --> JPA API (javax/jakarta.persistence) --> Hibernate Implementation --
> JDBC --> Database
```

Hibernate 6.x Key Changes:

- Migrated from `javax.persistence` to `jakarta.persistence` (Jakarta EE namespace)
- New Semantic Query Model (SQM) replaces the old AST-based query parser
- Improved type system with better SQL type inference
- Built-in support for Java Records as DTOs

Why Hibernate Became Popular

1. **Eliminates boilerplate** -- No manual SQL, ResultSet mapping, connection management
2. **Database portability** -- Switch from MySQL to PostgreSQL without changing code
3. **Automatic schema generation** -- Create/update tables from entity classes
4. **Caching** -- Built-in first-level cache, pluggable second-level cache
5. **Lazy loading** -- Load associated data only when accessed
6. **Dirty checking** -- Automatically detects and persists state changes

How Spring Boot Auto-Configures Hibernate

When you add `spring-boot-starter-data-jpa` to your project, Spring Boot automatically:

```
1. Detects Hibernate on classpath (it ships with the starter)
2. Creates HikariCP DataSource from application.yml properties
3. Creates LocalContainerEntityManagerFactoryBean
4. Scans for @Entity classes in your base package
5. Builds SessionFactory (wrapped as EntityManagerFactory)
6. Creates PlatformTransactionManager (JpaTransactionManager)
7. Registers Spring Data JPA repositories
```

Result: Zero manual Hibernate configuration needed. You just write entities and repositories.

2.2 How Hibernate Works Internally

```
+---------------------------------------------+
|           Java Application                  |
|  (Entities, Services, Repositories)         |
+---------------------------------------------+
         |  persist()  find()  merge()
         v
+---------------------------------------------+
|         Hibernate ORM Layer                 |
|  +------------+  +-------------+            |
|  | Session    |  | Transaction |            |
|  | Factory    |  | Manager     |            |
|  +------------+  +-------------+            |
```

```
|  +------------+  +-------------+            |
|  | 1st Level  |  | Query       |            |
|  | Cache      |  | Translator  |            |
|  +------------+  +-------------+            |
|  +------------+                             |
|  | Dialect    | (MySQL, PostgreSQL, Oracle)   |
|  +------------+                             |
+---------------------------------------------+
         |  SQL statements
         v
+---------------------------------------------+
|              JDBC Driver                    |
+---------------------------------------------+
         |
         v
+---------------------------------------------+
|        Relational Database                  |
|   (MySQL, PostgreSQL, Oracle, H2)           |
+---------------------------------------------+
```

Internal Flow: Saving an Entity

```
1. Application calls entityManager.persist(user)
2. Hibernate assigns ID (via strategy)
3. Entity added to Persistence Context (1st level cache)
4. Entity marked as "dirty" (new)
5. On transaction commit / flush:
   a. Hibernate generates INSERT SQL using Dialect
   b. SQL sent to JDBC driver
   c. JDBC executes against database
   d. Entity state changed to "persistent"
```

Deep Dive: What Happens Inside persist()

```
entityManager.persist(user)
  |
  v
SessionImpl.persist(user)
  |
  v
ActionQueue.addAction(EntityInsertAction)
  |-- Entity added to Persistence Context (IdentityMap)
  |-- Snapshot of entity state captured
```

```
  |-- ID generated (depends on strategy):
  |     IDENTITY -> Must execute INSERT immediately to get auto-generated ID
  |     SEQUENCE -> Pre-allocates IDs from sequence (batch-friendly)
  |     UUID     -> Generated in Java, no DB call needed
  |
  v
On flush() / commit():
  |
  v
ActionQueue.executeActions()
  |-- Sorts actions by entity type (for batching)
  |-- Generates SQL via Dialect
  |-- Executes via PreparedStatement (JDBC)
  |-- Updates entity state to MANAGED
```

Why IDENTITY disables batch inserts: With `GenerationType.IDENTITY`, Hibernate must execute an INSERT immediately upon `persist()` to retrieve the auto-generated ID from the database. This breaks batching because each INSERT must be executed individually. Use `SEQUENCE` with `allocationSize=50` for batch-friendly ID generation.

Monolithic vs Microservices: Hibernate Configuration

Aspect	Monolithic	Microservices
SessionFactory	One per application	One per service
Database	Single shared database	Database per service
Transactions	Single `@Transactional` spans all tables	Each service manages its own transactions
Entity relationships	Full JPA relationships across all entities	NO cross-service entity relationships
Data consistency	ACID (single DB transaction)	Eventual consistency (Saga pattern)

Interview Questions for Part 2

Q: What is the difference between JPA and Hibernate? A: JPA is a specification (set of interfaces and annotations). Hibernate is an implementation that provides the actual code behind those interfaces. Your application code should depend on JPA interfaces, while Spring Boot configures Hibernate as the JPA provider behind the scenes.

Q: Why does GenerationType.IDENTITY disable batch inserts? A: Because IDENTITY relies on database auto-increment. Hibernate must execute each INSERT individually to retrieve the generated ID. With SEQUENCE, Hibernate can pre-allocate a block of IDs (allocationSize=50) and batch 50 INSERTs in a single JDBC call.

Part 3: Hibernate Core Architecture

3.1 SessionFactory

What: A heavyweight, thread-safe, immutable factory for creating Sessions. Built once at application startup from configuration.

Why: Creating database connections is expensive. SessionFactory pre-configures mappings, caches, and connection pools.

When: One per database per application (singleton).

Deep Theory: What SessionFactory Contains Internally

When Hibernate builds a SessionFactory at startup, it performs expensive one-time initialization:

```
SessionFactory contains:
  |
  +-- Entity Metamodel (compiled @Entity mappings)
  |     Each entity class -> table name, column mappings, relationship graph
  |
  +-- Named Query Cache (pre-compiled @NamedQuery / @Query definitions)
  |     Queries validated at startup (fail-fast if JPQL has errors)
  |
  +-- Connection Pool Reference (HikariCP DataSource)
  |     Manages physical database connections
  |
  +-- Second-Level Cache Configuration (if enabled)
  |     Cache regions per entity type
  |
  +-- Dialect Instance
  |     Database-specific SQL generation rules
  |
  +-- Type Registry
        Java type -> SQL type mappings
```

Production Impact: Because SessionFactory is so expensive to build (parses all entities, validates all named queries, connects to database), it should NEVER be created per-request. A misconfigured Spring application that accidentally creates multiple SessionFactories will consume excessive memory and slow startup dramatically.

How to detect: If your Spring Boot app takes 30+ seconds to start, check for multiple `EntityManagerFactory` beans or duplicate `@EntityScan` configurations.

```
// Spring Boot auto-configures this -- you rarely create it manually
// But understanding it is critical for interviews

// Manual creation (non-Spring):
Configuration config = new Configuration()
    .configure("hibernate.cfg.xml")
    .addAnnotatedClass(User.class);
SessionFactory factory = config.buildSessionFactory();

// In Spring Boot: auto-configured as EntityManagerFactory
@Autowired
private EntityManagerFactory entityManagerFactory;
// EntityManagerFactory wraps SessionFactory (JPA standard interface)
```

Production Scenario: Multiple DataSources

In a monolithic app that reads from a primary DB and a legacy DB:

```
// Primary SessionFactory
@Primary
@Bean
public LocalContainerEntityManagerFactoryBean primaryEMF(DataSource primaryDS)
{
    // ... configured for primary database
}

// Legacy SessionFactory
@Bean
public LocalContainerEntityManagerFactoryBean legacyEMF(DataSource legacyDS) {
    // ... configured for legacy read-only database
}
// Each SessionFactory = separate persistence unit = separate 1st/2nd level
caches
```

3.2 Session (EntityManager)

What: A lightweight, short-lived, non-thread-safe object representing a unit of work. Wraps a JDBC connection. **JPA equivalent: EntityManager.**

Why: Provides the API for CRUD operations and serves as the first-level cache (persistence context).

When: One per request/transaction (request-scoped in web applications).

Deep Theory: Why Session is NOT Thread-Safe

The Session maintains an internal `IdentityMap` (the first-level cache) that maps entity IDs to object references. This map is NOT synchronized. If two threads share a Session:

- Thread A loads User(id=1), puts it in the IdentityMap
- Thread B loads User(id=1), gets Thread A's reference
- Thread A modifies User.name = "Alice"
- Thread B reads User.name -- sees "Alice" (corrupted data!)

Spring Boot's solution: Spring wraps the EntityManager in a thread-bound proxy. Each HTTP request thread gets its own EntityManager via `@PersistenceContext`.

```
@PersistenceContext
private EntityManager em; // This is a PROXY, not the actual EntityManager
// Spring creates a real EntityManager per thread from the SessionFactory
```

Deep Dive: The Persistence Context

The Persistence Context is the "heart" of Hibernate. It is a Map inside each Session:

```
Persistence Context (inside Session):
  Key: EntityKey(type=User.class, id=1)  ->  Value: User@abc (Java object
reference)
  Key: EntityKey(type=User.class, id=2)  ->  Value: User@def
  Key: EntityKey(type=Order.class, id=5) ->  Value: Order@ghi

Also stores:
  - Entity snapshots (loaded state for dirty checking)
  - Pending actions queue (INSERT, UPDATE, DELETE)
  - Collection entries (lazy proxy references)
```

```
// Session operations
Session session = sessionFactory.openSession();

// JPA equivalent (what Spring uses)
EntityManager em = entityManagerFactory.createEntityManager();
```

```
// CRUD operations
em.persist(user);              // INSERT
User u = em.find(User.class, 1L);  // SELECT by ID
em.merge(user);                // UPDATE
em.remove(user);               // DELETE
```

Production Problem: Bloated Persistence Context

Scenario: A batch job loads 100,000 entities for processing:

```
// BAD: Persistence Context grows to 100K entities -> OutOfMemoryError
@Transactional
public void processAll() {
    List<User> users = userRepository.findAll(); // 100K entities loaded!
    users.forEach(u -> u.setProcessed(true));
    // At flush: Hibernate dirty-checks ALL 100K entities -> extremely slow
}

// GOOD: Process in batches, clear persistence context periodically
@Transactional
public void processAll() {
    int page = 0;
    Page<User> batch;
    do {
        batch = userRepository.findAll(PageRequest.of(page, 500));
        batch.forEach(u -> u.setProcessed(true));
        entityManager.flush();  // Execute pending UPDATEs
        entityManager.clear();  // Clear 1st level cache -> free memory
        page++;
    } while (batch.hasNext());
}
```

3.3 Transaction

What: Wraps a database transaction. Ensures ACID properties.

Deep Theory: How @Transactional Works Under the Hood

```
1. Spring creates a CGLIB proxy around your @Service class
2. When a @Transactional method is called from OUTSIDE the class:
   a. Proxy intercepts the method call
   b. TransactionInterceptor obtains a Connection from HikariCP
```

```
  c. Sets auto-commit = false on the Connection
  d. Binds Connection to current thread (ThreadLocal)
  e. Calls your actual method
  f. On success: Connection.commit()
  g. On RuntimeException: Connection.rollback()
  h. Returns Connection to pool
```

Critical Gotcha -- Self-Invocation:

```java
@Service
public class OrderService {

    @Transactional
    public void placeOrder(OrderRequest req) {
        // ...
        this.logAudit(req); // BUG! Self-invocation bypasses proxy!
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void logAudit(OrderRequest req) {
        // This does NOT run in a new transaction!
        // The proxy is bypassed when calling methods within the same class.
    }
}

// Fix: Inject self or extract to a separate class
@Autowired private OrderService self; // Inject proxy reference
self.logAudit(req); // Now goes through the proxy -> new transaction
```

```java
// Manual transaction (non-Spring)
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
try {
    session.persist(user);
    tx.commit();
} catch (Exception e) {
    tx.rollback();
    throw e;
} finally {
    session.close();
}

// Spring manages this with @Transactional -- much cleaner!
@Transactional
```

```
public void createUser(User user) {
    entityManager.persist(user); // Auto-commit or rollback
}
```

3.4 Hibernate Dialect

What: Translates HQL/JPQL to database-specific SQL.

Deep Theory: Why Dialects Exist

Different databases implement SQL differently:

- **Pagination:** MySQL uses `LIMIT 10 OFFSET 20`, Oracle uses `FETCH FIRST 10 ROWS ONLY`, SQL Server uses `TOP 10`
- **ID Generation:** PostgreSQL has `SEQUENCE`, MySQL uses `AUTO_INCREMENT`
- **Data Types:** PostgreSQL has `TEXT`, `JSONB`, `UUID`; MySQL uses `LONGTEXT`, `JSON`
- **Locking:** PostgreSQL uses `SELECT ... FOR UPDATE SKIP LOCKED`, Oracle syntax differs

The Hibernate Dialect translates your database-agnostic JPQL into the correct SQL for your target database. This is how you achieve **database portability** -- the same Java code runs on any supported database.

Production Scenario -- Database Migration: A healthcare company migrates from Oracle to PostgreSQL to save licensing costs. With Hibernate, the migration involves:

1. Change `database-platform` in `application.yml`
2. Update JDBC URL and driver
3. Test all native queries (which are NOT portable)
4. Run Flyway migrations to recreate schema

NO changes to entity classes, repositories, or service logic.

Database	Dialect Class
MySQL 8	`org.hibernate.dialect.MySQLDialect`
PostgreSQL	`org.hibernate.dialect.PostgreSQLDialect`
Oracle	`org.hibernate.dialect.OracleDialect`
H2	`org.hibernate.dialect.H2Dialect`
SQL Server	`org.hibernate.dialect.SQLServerDialect`

```yaml
# application.yml
spring:
  jpa:
    database-platform: org.hibernate.dialect.PostgreSQLDialect
    properties:
      hibernate:
        format_sql: true
        show_sql: true
```

Interview Questions for Part 3

Q: What is the difference between SessionFactory and Session? A: SessionFactory is heavyweight, thread-safe, created once at startup (singleton). It holds compiled entity mappings, caches, and connection pool config. Session is lightweight, non-thread-safe, created per-request/transaction. Session IS the persistence context (first-level cache) and wraps a JDBC connection.

Q: Why is Session NOT thread-safe? How does Spring handle this? A: Session maintains an unsynchronized IdentityMap. Spring injects a thread-bound proxy via `@PersistenceContext`. Each HTTP request thread gets its own real EntityManager, ensuring thread isolation.

Q: What happens if you accidentally create two SessionFactory instances? A: You get double memory consumption (two metamodels, two caches), potential data inconsistency (entity loaded in SessionFactory-1 is invisible to SessionFactory-2's cache), and slow startup. This is a common misconfiguration bug.

Part 4: Entity Mapping

4.1 Basic Annotations

Deep Theory: What Makes a Valid @Entity

Hibernate requires specific conditions for an `@Entity` class:

1. Must be a **top-level class** or a `static` inner class (non-static inner classes can't be instantiated via reflection)
2. Must have a **no-argument constructor** (can be `protected` -- Hibernate uses reflection)
3. Must NOT be `final` (Hibernate creates CGLIB proxies for lazy loading)
4. Must have an `@Id` field (every entity needs a primary key)
5. Should implement `Serializable` if used in second-level cache or distributed sessions

Production Bug -- Final Entity:

```
@Entity
public final class Product { ... } // BUG! Hibernate cannot create lazy proxy
// Result: LazyInitializationException or forced EAGER loading
```

@Entity

Marks a Java class as a persistent entity mapped to a database table.

```
@Entity // Required -- tells Hibernate "this is a database table"
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name; // Maps to column "name" by default
}
```

@Table

Customizes the table name and constraints.

```java
@Entity
@Table(
    name = "products",
    schema = "ecommerce",
    uniqueConstraints = @UniqueConstraint(columnNames = {"sku", "region"}),
    indexes = @Index(name = "idx_product_name", columnList = "name")
)
public class Product { ... }
```

Why @Index matters in production: Without database indexes on frequently queried columns, SELECT queries scan the entire table. A table with 10 million rows and no index on `name` takes 2-5 seconds per query. With an index: < 5 milliseconds.

@Id and @GeneratedValue

Strategy	Description	Database	Use When
`IDENTITY`	Auto-increment column	MySQL, PostgreSQL	Single DB, simple apps
`SEQUENCE`	Database sequence	PostgreSQL, Oracle	High-performance batch inserts
`TABLE`	Uses a separate table	All	Portable but slow
`UUID`	UUID generation	All	Distributed systems
`AUTO`	Hibernate picks best	Varies	Default (not recommended)

Deep Dive: Why SEQUENCE is Superior for Performance

```
IDENTITY strategy:
  persist(user1) -> INSERT INTO users ... (must execute NOW to get ID) -> id=1
  persist(user2) -> INSERT INTO users ... (must execute NOW to get ID) -> id=2
  persist(user3) -> INSERT INTO users ... (must execute NOW to get ID) -> id=3
  Result: 3 individual INSERT statements (no batching possible)

SEQUENCE strategy (allocationSize=50):
  persist(user1) -> SELECT nextval('user_seq') -> returns 1 (allocates IDs 1-50)
  persist(user2) -> Uses pre-allocated ID 2 (no DB call!)
  persist(user3) -> Uses pre-allocated ID 3 (no DB call!)
  ... on flush:
  Result: Single batch INSERT for all 3 users (JDBC batching enabled!)
```

Performance numbers: Inserting 10,000 entities:

- IDENTITY: ~10,000 individual INSERTs = 8-12 seconds
- SEQUENCE (allocationSize=50): ~200 batch INSERTs = 0.5-1 second (10x faster)

```
// IDENTITY (MySQL, PostgreSQL auto-increment)
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

// SEQUENCE (PostgreSQL, Oracle -- preferred for performance)
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE,
    generator = "product_seq")
@SequenceGenerator(name = "product_seq",
    sequenceName = "product_sequence",
    allocationSize = 50) // Pre-allocate 50 IDs -- huge performance gain
private Long id;

// UUID (distributed systems)
@Id
@GeneratedValue(strategy = GenerationType.UUID)
private UUID id;
```

When to use UUID: In microservices where multiple services insert into their own databases, and IDs must be globally unique. UUID eliminates the need for a centralized sequence. Tradeoff: UUIDs are 128-bit (larger index), and random UUIDs cause B-tree fragmentation. Use ordered UUIDs (UUIDv7) to mitigate.

@Column

```
@Column(
    name = "product_name",    // Column name (default: field name)
    nullable = false,         // NOT NULL constraint
    unique = true,            // UNIQUE constraint
    length = 150,             // VARCHAR length (default 255)
    precision = 10,           // Decimal precision
    scale = 2,                // Decimal scale
    columnDefinition = "TEXT" // Raw SQL column definition
)
private String name;
```

4.2 Composite Keys

When to Use Composite Keys

Use composite keys when: The business domain naturally identifies entities by multiple columns (e.g., a student's enrollment is identified by student_id + course_id). Common in legacy databases and many-to-many join tables with extra attributes.

Architect's preference: Most modern systems use a surrogate `@Id Long id` instead. Composite keys complicate queries, caching, and JPA/Spring Data integration.

```
// Method 1: @EmbeddedId
@Embeddable
public class OrderItemId implements Serializable {
    private Long orderId;
    private Long productId;

    // Constructors, equals(), hashCode() -- REQUIRED
}

@Entity
public class OrderItem {
    @EmbeddedId
    private OrderItemId id;

    private int quantity;
    private BigDecimal price;
}

// Method 2: @IdClass
@Entity
@IdClass(OrderItemId.class)
public class OrderItem {
    @Id private Long orderId;
    @Id private Long productId;
    private int quantity;
}
```

Critical: equals() and hashCode() are REQUIRED. Hibernate uses these for identity checking in the persistence context. If missing, two entities with the same composite key will be treated as different objects, causing data inconsistency.

4.3 Embedded Objects (Value Objects)

Deep Theory: Value Objects in Domain-Driven Design

An `@Embeddable` represents a **value object** -- something that has no identity of its own. An Address is not an independent entity; it exists only as part of a Customer. It has no lifecycle of its own and no primary key.

Architect's Rule: If it has NO independent identity, use `@Embeddable`. If it has its own lifecycle and is referenced by multiple entities, use `@Entity` with `@OneToOne`.

```
@Embeddable
public class Address {
    private String street;
    private String city;
    private String state;

    @Column(length = 10)
    private String zipCode;
}

@Entity
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street",
            column = @Column(name = "billing_street")),
        @AttributeOverride(name = "city",
            column = @Column(name = "billing_city"))
    })
    private Address billingAddress;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street",
            column = @Column(name = "shipping_street")),
        @AttributeOverride(name = "city",
            column = @Column(name = "shipping_city"))
    })
    private Address shippingAddress;
}
// Result: ONE table "customer" with columns:
// id, name, billing_street, billing_city, ..., shipping_street, shipping_city
```

Real-World Usage: E-commerce systems commonly embed `Money` (amount + currency), `DateRange` (start + end), and `GeoLocation` (lat + lng) as value objects.

4.4 Enums and Temporal Types

Deep Theory: EnumType.ORDINAL vs EnumType.STRING

NEVER use `EnumType.ORDINAL` in production! Ordinal stores the enum's position (0, 1, 2...). If someone reorders the enum constants or adds a new value between existing ones, ALL existing data becomes corrupt.

```
// DANGEROUS: ordinal values
public enum Status { ACTIVE, INACTIVE, SUSPENDED }
// ACTIVE=0, INACTIVE=1, SUSPENDED=2

// Someone adds a new status:
public enum Status { ACTIVE, PENDING, INACTIVE, SUSPENDED }
// Now: ACTIVE=0, PENDING=1, INACTIVE=2, SUSPENDED=3
// All rows with ordinal=1 (which were INACTIVE) now map to PENDING! DATA
CORRUPTION!
```

Always use `EnumType.STRING` -- stores the enum name as text. Immune to reordering.

```
@Entity
public class Employee {

    @Enumerated(EnumType.STRING) // Store "ACTIVE", not 0
    private EmployeeStatus status;

    @Column(updatable = false)
    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = createdAt;
    }

    @PreUpdate
    protected void onUpdate() {
```

```
        updatedAt = LocalDateTime.now();
    }
}
```

Interview Questions for Part 4

Q: Why should you NEVER use EnumType.ORDINAL? A: Because it stores the enum's positional index (0, 1, 2). If someone reorders the enum or inserts a new constant, all existing database values silently map to wrong enum constants, causing data corruption that's extremely difficult to detect and fix.

Q: Why does SEQUENCE strategy outperform IDENTITY for batch inserts? A: SEQUENCE pre-allocates a block of IDs (e.g., 50) in a single DB call, allowing Hibernate to assign IDs in-memory and batch multiple INSERTs in a single JDBC call. IDENTITY requires an individual INSERT per entity to retrieve the auto-generated ID.

Part 5: Hibernate Entity Lifecycle

5.1 Entity States

Deep Theory: Why Entity States Matter

Understanding entity states is the single most important concept for debugging Hibernate issues in production. Every "mysterious" bug -- unexpected UPDATEs, missing data, `LazyInitializationException`, stale data -- traces back to the entity being in the wrong state.

The key insight: Hibernate only tracks changes to **Persistent** entities. If your entity is Transient or Detached, modifications are silently ignored. This is the #1 source of "my save isn't working" bugs.

```
                new Entity()
                     |
                     v
            +-------------+
            |  TRANSIENT  |   Not managed, no ID, not in DB
            +-------------+
                     |
             persist()
                     |
                     v
            +-------------+
            | PERSISTENT  |   Managed by Session, has ID, in DB
            +-------------+   Changes auto-detected (dirty checking)
              |         |
      detach()/close()   remove()
              |         |
              v         v
        +----------+  +---------+
        | DETACHED |  | REMOVED |
        +----------+  +---------+
             |              Not managed, scheduled for DELETE
        merge()
             |
             v
        +-------------+
        | PERSISTENT  |   Re-attached to new Session
        +-------------+
```

5.2 State Transitions with Code

```
// TRANSIENT -> New object, not yet associated with Session
User user = new User("John", "john@email.com");
// user is TRANSIENT -- Hibernate doesn't know about it

// TRANSIENT -> PERSISTENT
entityManager.persist(user);
// user is now PERSISTENT -- managed by the persistence context
// Hibernate tracks all changes to this object

// PERSISTENT -- Dirty Checking
user.setEmail("john.updated@email.com");
// NO explicit save/update needed!
// Hibernate detects the change and generates UPDATE on flush/commit

// PERSISTENT -> DETACHED
entityManager.detach(user);
// or: entityManager.close()
// user is DETACHED -- no longer managed
user.setName("Jane"); // This change will NOT be auto-persisted

// DETACHED -> PERSISTENT (re-attach)
User managed = entityManager.merge(user);
// 'managed' is PERSISTENT; 'user' is still DETACHED
// Use the returned reference!

// PERSISTENT -> REMOVED
entityManager.remove(managed);
// Entity scheduled for DELETE on flush/commit
```

Production Bug: merge() Returns a NEW Reference

```
// COMMON BUG:
User detached = getDetachedUser();
entityManager.merge(detached);
detached.setName("Updated"); // BUG! 'detached' is still DETACHED!

// CORRECT:
User managed = entityManager.merge(detached);
managed.setName("Updated"); // 'managed' is PERSISTENT -- change tracked!
```

Deep Dive: How Dirty Checking Works Internally

```
1. When entity is loaded (find/query), Hibernate stores a SNAPSHOT:
   snapshot = { name: "John", email: "john@email.com", status: "ACTIVE" }

2. Your code modifies the entity:
   user.setEmail("john.new@email.com")

3. On flush(), Hibernate compares EVERY field:
   current.name == snapshot.name?  YES -> skip
   current.email == snapshot.email? NO  -> mark dirty
   current.status == snapshot.status? YES -> skip

4. Only dirty fields generate SQL:
   UPDATE users SET email = 'john.new@email.com' WHERE id = 1
   (NOT: UPDATE users SET name='John', email='john.new@email.com',
status='ACTIVE' WHERE id = 1)
```

Performance concern: Dirty checking compares ALL fields of ALL persistent entities on every flush. With 10,000 entities in the persistence context, each having 20 fields, that's 200,000 comparisons per flush. This is why `@Transactional(readOnly = true)` is a performance optimization -- it tells Hibernate to skip dirty checking entirely.

@DynamicUpdate: By default, Hibernate generates UPDATE statements with ALL columns (for statement caching efficiency). Use `@DynamicUpdate` on entities with many columns to generate UPDATEs with only changed columns, reducing database I/O.

5.3 Important Details

State	In Session?	Has ID?	In DB?	Changes Tracked?
Transient	No	No	No	No
Persistent	Yes	Yes	Yes (or pending INSERT)	Yes (dirty checking)
Detached	No	Yes	Yes	No
Removed	Yes	Yes	Yes (pending DELETE)	No

FlushMode: When Does Hibernate Execute SQL?

FlushMode	Behavior	Use Case

`AUTO` (default)	Flush before queries and on commit	Most applications
`COMMIT`	Flush only on commit	Performance optimization when queries don't depend on pending changes
`MANUAL`	Flush only when explicitly called	Full control (advanced)

Production Tip: If you're executing a JPQL `SELECT` and Hibernate flushes pending INSERTs before it (to ensure query consistency), this AUTO flush can be unexpectedly slow. Use `readOnly = true` for pure read transactions to avoid unnecessary flushes.

Interview Questions for Part 5

Q: What happens if you call entityManager.merge() and then modify the ORIGINAL (not returned) reference? A: The original object remains DETACHED. Only the returned reference is PERSISTENT. Modifying the original has no effect on the database. This is a very common bug.

Q: Why does @Transactional(readOnly=true) improve performance? A: Three reasons: (1) Hibernate skips dirty checking on all loaded entities (no snapshot comparison), (2) Hibernate may skip entity snapshots entirely (less memory), (3) Spring may route the query to a database read replica.

Part 6: Associations in Hibernate

Deep Theory: How Hibernate Maps Relationships to SQL

Hibernate translates Java object references into SQL foreign keys and join tables. Understanding the SQL generated by each annotation is critical for debugging performance issues.

Architect's Rule: Always think in terms of SQL first, then map to JPA annotations. If you don't know what SQL will be generated, you'll create N+1 problems, unnecessary JOINs, and cascade disasters.

6.1 @ManyToOne (Most Common)

Why @ManyToOne is the Most Important Annotation

In most enterprise systems, 80% of relationships are @ManyToOne. An Order belongs to a Customer. A Product belongs to a Category. An Employee belongs to a Department. Understanding this annotation deeply is essential.

```java
@Entity
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY) // LAZY is best practice!
    @JoinColumn(name = "customer_id", nullable = false)
    private Customer customer;

    private BigDecimal totalAmount;
}
// DB: orders table has customer_id FK column
```

What Hibernate generates:

```table_of_contents
CREATE TABLE orders (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    customer_id BIGINT NOT NULL,
    total_amount DECIMAL(19,2),
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

Critical: Always set `fetch = FetchType.LAZY` on @ManyToOne! The JPA default for @ManyToOne is EAGER. This means EVERY time you load an Order, Hibernate also loads the Customer via a JOIN, even if you never access `order.getCustomer()`. In a list of 1000 orders, this generates 1000 unnecessary Customer loads.

6.2 @OneToMany (Inverse Side)

Deep Theory: The Inverse Side Does NOT Control the Relationship

The inverse side (`mappedBy`) is a mirror. It tells Hibernate "the relationship is already mapped by the other side." Changes to the inverse side's collection are IGNORED by Hibernate for persistence purposes.

```
// THIS DOES NOT WORK:
Customer customer = new Customer();
customer.getOrders().add(new Order()); // Order has NO customer reference
entityManager.persist(customer);
// Result: Order is saved with customer_id = NULL!

// THIS WORKS:
Order order = new Order();
order.setCustomer(customer); // SET THE OWNING SIDE
entityManager.persist(order);
// Result: Order is saved with correct customer_id FK
```

Best Practice: Always use helper methods to keep both sides in sync:

```
@Entity
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "customer", // Inverse side -- no FK column here
               cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Order> orders = new ArrayList<>();

    // Helper methods to maintain bidirectional consistency
    public void addOrder(Order order) {
        orders.add(order);
        order.setCustomer(this);
    }
```

```
    public void removeOrder(Order order) {
        orders.remove(order);
        order.setCustomer(null);
    }
}
```

6.3 @OneToOne

Production Warning: @OneToOne Lazy Loading Trap

`@OneToOne` with `mappedBy` (inverse side) **cannot be lazy-loaded** by default! Hibernate cannot determine whether the related entity exists without querying the database. It must either load the entity (EAGER) or use bytecode enhancement.

```
@Entity
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "profile_id", unique = true)
    private UserProfile profile;
}

@Entity
public class UserProfile {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(mappedBy = "profile") // WARNING: Cannot be lazy on the inverse
side!
    private User user;

    private String bio;
    private String avatarUrl;
}
```

Fix for inverse @OneToOne lazy loading:

- Use `@MapsId` instead (share the primary key)
- Use bytecode enhancement (`hibernate.enhance_lazy_loading=true`)

- Or accept EAGER loading on the inverse side

6.4 @ManyToMany

Architect's Warning: Avoid @ManyToMany in Production

In real enterprise systems, pure `@ManyToMany` is almost never the right choice. The join table inevitably needs extra columns (enrollment_date, grade, status). You should use an explicit join entity instead.

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany
    @JoinTable(
        name = "student_courses",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();
}

@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students = new HashSet<>();
}
```

Why Set instead of List for @ManyToMany: Using `List` with @ManyToMany causes Hibernate to delete ALL rows in the join table and re-insert them every time the collection is modified. Using `Set` allows Hibernate to add/remove individual rows. This is a critical performance difference with large datasets.

Better approach with explicit join entity:

```
@Entity
public class Enrollment {
```

```
    @EmbeddedId
    private EnrollmentId id;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("studentId")
    private Student student;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("courseId")
    private Course course;

    private LocalDate enrolledDate;
    private String grade;
}
```

6.5 Cascade Types

Cascade Type	Effect	When to Use
`PERSIST`	Save child when parent is saved	Parent-child relationships
`MERGE`	Update child when parent is updated	Almost always with PERSIST
`REMOVE`	Delete child when parent is deleted	Owned relationships
`REFRESH`	Refresh child when parent is refreshed	Rare
`DETACH`	Detach child when parent is detached	Rare
`ALL`	All of the above	Strong parent-child ownership

Production Danger: CascadeType.ALL Can Delete Your Data

```
// DANGEROUS: CascadeType.ALL includes REMOVE
@ManyToOne(cascade = CascadeType.ALL) // NEVER cascade on @ManyToOne!
@JoinColumn(name = "department_id")
private Department department;

// If you delete an Employee, CascadeType.ALL cascades REMOVE to Department
// which deletes the Department AND all other employees in it!
```

Rule: NEVER use `cascade` on `@ManyToOne`. Only cascade from parent to child (OneToMany), never from child to parent.

6.6 Owning Side vs Inverse Side

Concept	Owning Side	Inverse Side
Has the FK	Yes (JoinColumn)	No
Controls relationship	Yes	No -- uses `mappedBy`
Annotation	`@JoinColumn`	`mappedBy = "fieldName"`
Example	`Order.customer` (has customer_id FK)	`Customer.orders` (no FK)

Rule: The side with the foreign key is always the owning side. In `@ManyToOne`, the "Many" side always owns.

Interview Questions for Part 6

Q: What happens if you add an Order to Customer.orders but don't set Order.customer? A: The order is saved with `customer_id = NULL` because only the owning side (Order.customer) controls the foreign key. Changes to the inverse side (Customer.orders) are ignored for SQL generation.

Q: Why should you avoid @ManyToMany with List? A: Because Hibernate deletes ALL rows in the join table and re-inserts them on any modification. Using Set allows individual row add/remove operations, which is dramatically more efficient.

Q: Can @OneToOne with mappedBy be lazy-loaded? A: No, not by default. Hibernate can't determine if the related entity exists without a query. Solutions: use @MapsId (shared primary key), bytecode enhancement, or accept EAGER loading on the inverse side.

Part 7: Fetching Strategies

Deep Theory: Why Fetching Strategy is the #1 Performance Decision

Every Hibernate performance problem in production traces back to incorrect fetching. Loading too much data (EAGER) wastes memory and bandwidth. Loading too little (LAZY without planning) causes the N+1 problem. The architect's job is to choose the right strategy for each use case.

Architect's Rule: Define entity associations as LAZY, then selectively fetch eagerly per-query using JOIN FETCH or @EntityGraph. NEVER use `FetchType.EAGER` on the entity definition.

7.1 Lazy vs Eager Loading

Lazy Loading (Default for Collections)

```
@OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
private List<Order> orders;
// SQL: Only loads Customer. Orders loaded ONLY when accessed.
// SELECT * FROM customers WHERE id = ?
// ... later when customer.getOrders() is called:
// SELECT * FROM orders WHERE customer_id = ?
```

How Lazy Loading Works Internally

When you define a LAZY association, Hibernate doesn't load the data. Instead, it creates a **proxy object** (or PersistentCollection for collections). The proxy looks like a real object but contains no data. When you call a method on it (e.g., `orders.size()`), Hibernate intercepts the call, executes the SELECT, and populates the data.

```
customer.getOrders() returns:
  PersistentBag (Hibernate proxy collection)
  |-- initialized = false
  |-- session = Session@xyz (reference to current session)

customer.getOrders().size() triggers:
  1. Proxy detects it's not initialized
  2. Checks if Session is still open (if NOT -> LazyInitializationException!)
  3. Executes: SELECT * FROM orders WHERE customer_id = ?
  4. Populates the collection
```

```
  5. Sets initialized = true
  6. Returns actual size
```

Eager Loading

```java
@ManyToOne(fetch = FetchType.EAGER)
private Customer customer;
// SQL: Loads Customer AND related data immediately via JOIN
// SELECT c.*, o.* FROM customers c JOIN orders o ON ...
```

Defaults

Association	Default Fetch Type
`@ManyToOne`	EAGER
`@OneToOne`	EAGER
`@OneToMany`	LAZY
`@ManyToMany`	LAZY

Best Practice: Always set `@ManyToOne(fetch = FetchType.LAZY)` and `@OneToOne(fetch = FetchType.LAZY)`. Override defaults.

7.2 The N+1 Query Problem

The most common Hibernate performance problem in production.

Root Cause Analysis

The N+1 problem occurs when:

1. You load a list of parent entities (1 query)
2. For each parent, Hibernate lazily loads a child collection (N queries)
3. Total: 1 + N queries

Why it's dangerous: With 100 customers, you get 101 queries. With 10,000 customers, you get 10,001 queries. Each query has network round-trip overhead (1-5ms). 10,001 queries x 2ms = 20 seconds just in network latency!

```
// N+1 Problem Example:
List<Customer> customers = customerRepository.findAll();
// Query 1: SELECT * FROM customers  (returns 100 customers)

for (Customer c : customers) {
    System.out.println(c.getOrders().size());
    // Query 2: SELECT * FROM orders WHERE customer_id = 1
    // Query 3: SELECT * FROM orders WHERE customer_id = 2
    // ...
    // Query 101: SELECT * FROM orders WHERE customer_id = 100
}
// TOTAL: 1 + 100 = 101 queries! Extremely slow!
```

Solution 1: JOIN FETCH (JPQL)

```
@Query("SELECT c FROM Customer c JOIN FETCH c.orders")
List<Customer> findAllWithOrders();
// Single query: SELECT c.*, o.* FROM customers c JOIN orders o ON ...
```

Warning -- Cartesian Product Problem: If you JOIN FETCH two collections simultaneously, you get a Cartesian product. Customer with 5 orders and 3 addresses = 15 result rows per customer!

```
// BAD: Cartesian product (MultipleBagFetchException with Lists)
@Query("SELECT c FROM Customer c JOIN FETCH c.orders JOIN FETCH c.addresses")
List<Customer> findAllWithOrdersAndAddresses(); // EXCEPTION!

// Fix: Use Set instead of List for one collection, OR fetch in separate
queries
```

Solution 2: @EntityGraph

```
@EntityGraph(attributePaths = {"orders"})
List<Customer> findAll();
// Generates LEFT JOIN FETCH automatically
```

Solution 3: @BatchSize

```
@OneToMany(mappedBy = "customer")
@BatchSize(size = 25)
private List<Order> orders;
// Instead of 100 queries, fires 4 queries (100/25 = 4 batches):
// SELECT * FROM orders WHERE customer_id IN (1,2,3,...,25)
// SELECT * FROM orders WHERE customer_id IN (26,27,...,50)
// ...
```

When to use @BatchSize: When you can't use JOIN FETCH (e.g., pagination with JOIN FETCH fails because Hibernate applies LIMIT in-memory). @BatchSize works correctly with pagination.

How to Detect N+1 in Production

```
# Enable Hibernate statistics
spring:
  jpa:
    properties:
      hibernate:
        generate_statistics: true
# Check logs for:
# Session Metrics { 101 JDBC statements prepared }
# If statement count is much higher than expected, you have N+1!
```

Interview Questions for Part 7

Q: What causes the N+1 problem? A: Loading N parent entities in one query, then lazily loading a child collection for each parent triggers N additional queries. Total = 1 + N. Fix with JOIN FETCH, @EntityGraph, or @BatchSize.

Q: Why can't you use JOIN FETCH with pagination? A: When you JOIN FETCH a collection and use Pageable, Hibernate can't apply SQL LIMIT/OFFSET correctly (each parent has multiple rows from the JOIN). Hibernate falls back to in-memory pagination, loading ALL data first. Use @BatchSize or DTO projections instead.

Part 8: Hibernate Query Methods

Deep Theory: How Hibernate Translates Queries

Hibernate provides three query languages. Understanding when to use each and what SQL they generate is crucial:

```
JPQL/HQL Query String
   |  (parsed by Hibernate Query Parser / SQM in Hibernate 6)
   v
SQL Abstract Syntax Tree (AST)
   |  (Dialect applies DB-specific rules)
   v
Native SQL String -> JDBC PreparedStatement -> Database
```

8.1 HQL (Hibernate Query Language)

```java
// HQL operates on ENTITIES, not tables
// Hibernate-specific (not portable to other JPA providers)

// Select
List<User> users = session.createQuery(
    "FROM User u WHERE u.status = :status", User.class)
    .setParameter("status", UserStatus.ACTIVE)
    .getResultList();

// Update
int updated = session.createQuery(
    "UPDATE User u SET u.status = :newStatus WHERE u.lastLogin < :cutoff")
    .setParameter("newStatus", UserStatus.INACTIVE)
    .setParameter("cutoff", LocalDateTime.now().minusDays(90))
    .executeUpdate();
```

Production Tip: Bulk UPDATE Bypasses Persistence Context

When you use HQL/JPQL `UPDATE` or `DELETE`, Hibernate executes SQL directly without updating the persistence context. Entities in memory become stale!

```
@Transactional
public void deactivateUsers() {
    entityManager.createQuery("UPDATE User u SET u.status = 'INACTIVE' WHERE
...")
        .executeUpdate();
    entityManager.clear(); // CRITICAL! Without this, cached entities still
show old status
}
```

8.2 JPQL (Java Persistence Query Language)

```
// JPQL is the JPA standard -- portable across ORM providers
// Very similar to HQL

@Query("SELECT u FROM User u WHERE u.email LIKE %:domain")
List<User> findByEmailDomain(@Param("domain") String domain);

@Query("SELECT NEW com.app.dto.UserSummaryDTO(u.id, u.name, u.email) " +
       "FROM User u WHERE u.status = :status")
List<UserSummaryDTO> findUserSummaries(@Param("status") UserStatus status);

// Aggregations
@Query("SELECT u.department, COUNT(u), AVG(u.salary) " +
       "FROM User u GROUP BY u.department")
List<Object[]> getDepartmentStats();
```

Best Practice: Use DTO Projections for Read-Only Queries

Loading full entities for display-only purposes wastes resources:

- Loads ALL columns (including large TEXT/BLOB fields)
- Creates entity snapshots (memory overhead for dirty checking)
- Loads EAGER associations unnecessarily

DTO projections skip ALL of this overhead.

8.3 Native SQL

```
// Use actual SQL -- not portable, but supports DB-specific features
@Query(value = "SELECT * FROM users WHERE email LIKE %?1 " +
               "ORDER BY created_at DESC LIMIT 10",
       nativeQuery = true)
List<User> findRecentByEmailDomain(String domain);

// With result mapping
@Query(value = "SELECT u.id, u.name, COUNT(o.id) as order_count " +
               "FROM users u LEFT JOIN orders o ON u.id = o.user_id " +
               "GROUP BY u.id, u.name",
       nativeQuery = true)
List<Object[]> getUserOrderCounts();
```

When to Use Native SQL in Production

Use Case	Why Native SQL
Window functions	`ROW_NUMBER() OVER(PARTITION BY ...)` not supported in JPQL
Common Table Expressions (CTEs)	`WITH ... AS (...)` not in JPQL
Full-text search	PostgreSQL `tsvector`, MySQL `MATCH AGAINST`
Database-specific JSON	PostgreSQL `jsonb_extract_path_text`
Performance-critical queries	Hand-optimized SQL with query hints

8.4 Comparison

Feature	HQL	JPQL	Native SQL
Standard	Hibernate-specific	JPA standard	Database-specific
Operates on	Entities	Entities	Tables
Portability	Hibernate only	Any JPA provider	Not portable
DB functions	Some	Limited	Full support
Complex queries	Good	Good	Best
Use when	Hibernate features needed	Default choice	Complex/optimized SQL

Interview Questions for Part 8

Q: What is the difference between HQL and JPQL? A: HQL is Hibernate-specific and supports Hibernate extensions. JPQL is the JPA standard, portable across JPA providers. In practice, most developers use JPQL via `@Query` in Spring Data repositories.

Q: Why do bulk HQL UPDATE/DELETE statements cause stale data? A: Because they execute SQL directly, bypassing the persistence context. Entities cached in the first-level cache still reflect old state. Fix: call `entityManager.clear()` or use `@Modifying(clearAutomatically = true)`.

Part 9: Transaction Management

What is Transaction Management in Spring?

Spring's `@Transactional` annotation provides **declarative transaction management** -- you annotate a method, and Spring automatically wraps it in a database transaction. No manual `begin/commit/rollback` code needed.

Why @Transactional Over Manual Transactions

Manual Transaction	@Transactional
10+ lines of try/catch/finally	Single annotation
Easy to forget rollback	Automatic rollback on exceptions
Scattered across codebase	Clean separation of concerns
Hard to test	Mockable, testable

Step-by-Step: How @Transactional Works Internally

```
1. Spring creates a proxy around the @Service class
2. When a @Transactional method is called:
   a. Proxy intercepts the call
   b. Opens a new transaction (or joins existing)
   c. Calls the actual method
   d. If method succeeds -> COMMIT
   e. If RuntimeException is thrown -> ROLLBACK
   f. If checked exception is thrown -> COMMIT (by default!)
```

> ***Critical Gotcha:*** *@Transactional does NOT work on* `private` *methods or when calling a @Transactional method from within the* ***same class*** *(self-invocation bypasses the proxy).*

Deep Theory: Checked vs Unchecked Exception Rollback

This catches even experienced developers off guard:

```
// RuntimeException (unchecked) -> ROLLS BACK (expected)
@Transactional
```

```java
public void method1() {
    userRepository.save(user);
    throw new RuntimeException("Error!"); // Transaction ROLLED BACK
}

// Checked Exception -> COMMITS! (unexpected!)
@Transactional
public void method2() throws IOException {
    userRepository.save(user);
    throw new IOException("Error!"); // Transaction COMMITTED! User is saved!
}

// Fix: Explicitly rollback for checked exceptions
@Transactional(rollbackFor = Exception.class) // Rolls back for ANY exception
public void method3() throws IOException {
    userRepository.save(user);
    throw new IOException("Error!"); // Now ROLLED BACK correctly
}
```

Best Practice: Always use `@Transactional(rollbackFor = Exception.class)` in production code.

9.1 ACID Properties

Property	Meaning	Example
Atomicity	All or nothing -- entire transaction succeeds or rolls back	Transfer $100: debit AND credit must both succeed
Consistency	Database moves from one valid state to another	Balance never goes negative if constraint exists
Isolation	Concurrent transactions don't interfere	Two users buying last item -- only one succeeds
Durability	Committed data survives crashes	After commit, data persists even if server restarts

Real-World ACID Scenario: Banking Transfer

```java
@Transactional(rollbackFor = Exception.class)
public void transfer(Long fromId, Long toId, BigDecimal amount) {
    Account from = accountRepo.findById(fromId)
        .orElseThrow(() -> new AccountNotFoundException(fromId));
```

```
    Account to = accountRepo.findById(toId)
        .orElseThrow(() -> new AccountNotFoundException(toId));

    from.debit(amount);   // Atomicity: if this succeeds but credit fails...
    to.credit(amount);    // ...both are rolled back (all or nothing)
    // Isolation: concurrent transfer uses @Version (optimistic lock)
    // Durability: once committed, even a server crash won't lose the data
}
```

9.2 Spring @Transactional

When to Use @Transactional

- **Service layer methods** that modify data (INSERT, UPDATE, DELETE)
- **Multi-step operations** where all steps must succeed or fail together
- Methods that read data and need **snapshot consistency** (`readOnly = true`)

When NOT to Use @Transactional

- **Controller methods** -- transactions should be scoped at the service layer
- **Very long-running operations** -- holding a transaction for minutes blocks database resources
- **External API calls** within a transaction -- the API call cannot be rolled back

Production Anti-Pattern: Calling External APIs Inside a Transaction

```
// BAD: External API call inside transaction
@Transactional
public void processOrder(OrderRequest req) {
    Order order = orderRepo.save(new Order(req));
    paymentGateway.charge(req.getPaymentInfo()); // Takes 2-5 seconds!
    // Transaction holds DB connection for 5+ seconds -> connection pool
exhaustion!
}

// GOOD: Separate transaction from external call
public void processOrder(OrderRequest req) {
    Order order = createOrder(req);              // @Transactional (fast)
    PaymentResult result = paymentGateway.charge(req); // No transaction
    updateOrderPayment(order.getId(), result);   // @Transactional (fast)
}
```

```java
@Service
public class OrderService {

    @Transactional // Wraps method in a database transaction
    public Order placeOrder(OrderRequest request) {
        // 1. Validate inventory
        Product product = productRepository.findById(request.getProductId())
            .orElseThrow(() -> new
ProductNotFoundException(request.getProductId()));

        if (product.getStock() < request.getQuantity()) {
            throw new InsufficientStockException();
        }

        // 2. Reduce stock
        product.setStock(product.getStock() - request.getQuantity());

        // 3. Create order
        Order order = new Order();
        order.setProduct(product);
        order.setQuantity(request.getQuantity());
        order.setTotalPrice(product.getPrice()
            .multiply(BigDecimal.valueOf(request.getQuantity())));

        // 4. Save and return
        return orderRepository.save(order);
        // If ANY step throws an exception, ALL changes are rolled back
    }
}
```

9.3 Transaction Propagation

What is Transaction Propagation?

Propagation defines what happens when a @Transactional method calls another @Transactional method. Should they share the same transaction? Create a new one? Throw an error?

Why This Matters in Production

Consider: An order placement method calls an audit logging method. If the order fails and rolls back, should the audit log also roll back? Usually NO -- you want the audit log to persist. This is where `REQUIRES_NEW` propagation saves you.

Propagation	Behavior	Use Case
`REQUIRED` (default)	Join existing or create new	Most service methods
`REQUIRES_NEW`	Always create new (suspend existing)	Audit logging that must persist even if parent fails
`SUPPORTS`	Join existing or run non-transactional	Read-only queries
`NOT_SUPPORTED`	Suspend existing, run non-transactional	Non-critical operations
`MANDATORY`	Must have existing, throw if none	Methods that should never be called directly
`NEVER`	Throw if transaction exists	Methods that must not run in a transaction
`NESTED`	Create savepoint within existing	Partial rollbacks

Deep Dive: REQUIRES_NEW vs NESTED

```java
// REQUIRES_NEW: Completely separate transaction
// If inner fails, outer can still commit
@Transactional
public void placeOrder(OrderRequest req) {
    Order order = orderRepo.save(new Order(req));
    try {
        notificationService.sendEmail(order); // REQUIRES_NEW
    } catch (Exception e) {
        log.error("Email failed, but order is still saved");
    }
}

// NESTED: Savepoint within SAME transaction
// If nested rolls back, outer can continue
// But if outer rolls back, ALL nested changes also roll back
@Transactional
public void importBatch(List<Record> records) {
    for (Record r : records) {
        try {
            processRecord(r); // NESTED: savepoint per record
        } catch (Exception e) {
            log.error("Record {} failed, skipping", r.getId());
        }
```

```
    }
}
```

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logAuditEvent(String action, String details) {
    // This runs in its OWN transaction
    // Even if the calling method rolls back, audit log is preserved
    AuditLog log = new AuditLog(action, details, Instant.now());
    auditRepository.save(log);
}
```

9.4 Isolation Levels

Deep Theory: What Each Isolation Problem Looks Like

Dirty Read: Transaction A reads uncommitted data from Transaction B. If B rolls back, A has read data that never existed.

Non-Repeatable Read: Transaction A reads a row, Transaction B modifies it and commits. A reads again and gets different values.

Phantom Read: Transaction A counts rows matching a condition. Transaction B inserts a new matching row. A counts again and gets a different count.

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
READ_UNCOMMITTED	Yes	Yes	Yes	Fastest
READ_COMMITTED	No	Yes	Yes	Good (PostgreSQL default)
REPEATABLE_READ	No	No	Yes	Medium (MySQL default)
SERIALIZABLE	No	No	No	Slowest

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public BigDecimal getAccountBalance(Long accountId) { ... }
```

Production Rule: Use your database's default isolation level unless you have a specific reason to change. Use `@Version` (optimistic locking) instead of SERIALIZABLE for concurrency control.

9.5 Read-Only Transactions

```
@Transactional(readOnly = true) // Performance optimization
public List<User> getAllActiveUsers() {
    return userRepository.findByStatus(UserStatus.ACTIVE);
    // Hibernate skips dirty checking -- faster!
    // Spring may route to read replica if configured
}
```

Deep Theory: Three Benefits of readOnly = true

1. **Hibernate optimization:** Skips dirty checking entirely (no entity snapshots stored, no field-by-field comparison at flush)
2. **Database optimization:** Some databases optimize read-only connections (PostgreSQL can avoid WAL overhead)
3. **Read replica routing:** Spring can route readOnly transactions to a database read replica using `AbstractRoutingDataSource`, reducing load on the primary

Interview Questions for Part 9

Q: Why does @Transactional NOT rollback on checked exceptions by default? A: By Spring design, checked exceptions are considered "recoverable" and don't trigger rollback. Use `@Transactional(rollbackFor = Exception.class)` to override.

Q: What happens if you call a @Transactional method from within the same class? A: The proxy is bypassed (self-invocation), so the second method runs without its own transaction. Fix: inject the bean into itself or extract to a separate service.

Q: When would you use REQUIRES_NEW propagation? A: For operations that must succeed independently, such as audit logging, notification tracking, or metrics. Even if the business operation rolls back, the audit log persists.

Part 10: Spring Data JPA Introduction

10.1 What is Spring Data JPA?

Spring Data JPA is an abstraction layer on top of JPA/Hibernate that eliminates boilerplate repository code. You define an interface, and Spring generates the implementation at runtime.

Why Spring Data JPA Exists

Without Spring Data JPA, every entity requires a repository class with 5+ methods of repetitive code (find, save, delete, update, count). Spring Data JPA generates these implementations **automatically** from just an interface declaration.

Design Patterns Used

Pattern	Where Used
Repository Pattern	Each interface represents a data access object for one entity
Proxy Pattern	Spring creates a runtime proxy implementing the interface
Template Method	`SimpleJpaRepository` provides default implementations
Strategy Pattern	Query derivation strategies (method name, @Query, native)

```java
// WITHOUT Spring Data JPA -- manual repository
@Repository
public class UserRepositoryImpl {

    @PersistenceContext
    private EntityManager em;

    public User findById(Long id) {
        return em.find(User.class, id);
    }

    public List<User> findByStatus(UserStatus status) {
        return em.createQuery(
            "SELECT u FROM User u WHERE u.status = :status", User.class)
            .setParameter("status", status)
            .getResultList();
    }
```

```java
    public User save(User user) {
        if (user.getId() == null) {
            em.persist(user);
            return user;
        }
        return em.merge(user);
    }

    public void delete(User user) {
        em.remove(em.contains(user) ? user : em.merge(user));
    }

    public List<User> findAll() {
        return em.createQuery("SELECT u FROM User u", User.class)
            .getResultList();
    }
}

// WITH Spring Data JPA -- just an interface!
public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByStatus(UserStatus status);
    // That's it! Spring generates everything else automatically.
}
```

10.2 How Spring Data JPA Works

1. You define an interface extending `JpaRepository`
2. Spring Boot scans for repository interfaces at startup
3. Spring Data creates a **proxy** implementation at runtime
4. The proxy delegates to `SimpleJpaRepository` (default implementation)
5. `SimpleJpaRepository` uses `EntityManager` internally

Deep Dive: What SimpleJpaRepository Does Internally

```java
// This is what Spring generates FOR YOU:
public class SimpleJpaRepository<T, ID> implements JpaRepository<T, ID> {

    private final EntityManager em;

    @Transactional
    public <S extends T> S save(S entity) {
        if (isNew(entity)) {
            em.persist(entity);   // New entity -> INSERT
```

```
            return entity;
        } else {
            return em.merge(entity); // Existing entity -> UPDATE
        }
    }

    // isNew() checks: is entity.getId() == null?
    // If yes -> persist(). If no -> merge().
    // This is why @GeneratedValue matters!
}
```

Production Gotcha -- save() on Entity with Preset ID: If you set the ID manually before calling `save()`, Spring thinks the entity already exists and calls `merge()` instead of `persist()`. This triggers a SELECT to check if the entity exists, then an INSERT -- resulting in 2 queries instead of 1. Fix: implement `Persistable<ID>` interface.

Interview Questions for Part 10

Q: How does Spring Data JPA decide whether to call persist() or merge() in save()? A: It checks `isNew()` which by default checks if the entity's `@Id` field is null. Null ID means new entity -> persist(). Non-null ID means existing -> merge(). Override this by implementing `Persistable<ID>`.

Part 11: Spring Data JPA Architecture

11.1 Architecture Diagram

```
+-----------------------------------------------------------------+
|                      Spring Boot Application                    |
|                                                                 |
|  +------------------+                                           |
|  |   Controller     |  @RestController                          |
|  |   (REST API)     |  Handles HTTP requests                    |
|  +--------+---------+                                           |
|           |                                                     |
|           v                                                     |
|  +------------------+                                           |
|  |    Service       |  @Service @Transactional                  |
|  |   (Business      |  Business logic and                       |
|  |    Logic)        |  transaction boundaries                   |
|  +--------+---------+                                           |
|           |                                                     |
|           v                                                     |
|  +------------------+                                           |
|  |   Repository     |  JpaRepository interface                  |
|  |   (Data Access)  |  Auto-generated implementation            |
|  +--------+---------+                                           |
|           |                                                     |
|           v                                                     |
|  +------------------+                                           |
|  | SimpleJpaRepo    |  Spring Data's default implementation     |
|  | (Generated Proxy)|  Uses EntityManager internally            |
|  +--------+---------+                                           |
|           |                                                     |
|           v                                                     |
|  +------------------+  +-------------------+                    |
|  | EntityManager    |  | Hibernate Session |                    |
|  | (JPA Standard)   |  | (Implementation)  |                    |
|  +--------+---------+  +-------------------+                    |
|           |                                                     |
|           v                                                     |
|  +------------------+                                           |
|  |   JDBC / HikariCP|  Connection pooling                      |
|  +--------+---------+                                           |
|           |                                                     |
|           v                                                     |
```

```
|  +------------------+                                          |
|  |   Database       |  PostgreSQL, MySQL, Oracle               |
|  +------------------+                                          |
+----------------------------------------------------------------+
```

11.2 The Stack

Layer	Technology	Role
API	Spring MVC / WebFlux	REST endpoints
Service	Spring @Service	Business logic, transactions
Repository	Spring Data JPA	Data access abstraction
ORM	Hibernate 6	Object-relational mapping
Connection Pool	HikariCP	JDBC connection management
Database	PostgreSQL/MySQL	Data storage

Deep Dive: Request Lifecycle Through the Stack

```
GET /api/users/1
  |
  v
Controller: userService.getUser(1)
  |
  v
Spring AOP Proxy intercepts @Transactional
  -> Obtains Connection from HikariCP pool
  -> Sets autocommit = false
  -> Binds Connection to ThreadLocal
  |
  v
Service calls userRepository.findById(1)
  |
  v
Spring Data proxy delegates to SimpleJpaRepository.findById()
  -> Calls entityManager.find(User.class, 1)
  |
  v
Hibernate checks 1st level cache (Persistence Context)
  -> Cache MISS -> generates SQL via Dialect
  -> SELECT u.* FROM users u WHERE u.id = ?
```

```
  |
  v
JDBC PreparedStatement executes on physical Connection
  -> ResultSet returned
  -> Hibernate maps to User entity
  -> Stores in Persistence Context (1st level cache)
  |
  v
Service returns UserDTO
  -> AOP Proxy calls Connection.commit()
  -> Connection returned to HikariCP pool
  |
  v
Controller returns ResponseEntity<UserDTO> as JSON
```

Part 12: Spring Data JPA Repositories

12.1 Repository Hierarchy

```
Repository (Marker interface)
    |
CrudRepository (CRUD operations)
    |
ListCrudRepository (List-returning CRUD)
    |
PagingAndSortingRepository (+ Pagination, Sorting)
    |
JpaRepository (+ JPA-specific: flush, batch, example queries)
```

12.2 CrudRepository

```java
// Provides basic CRUD
public interface CrudRepository<T, ID> extends Repository<T, ID> {
    <S extends T> S save(S entity);
    Optional<T> findById(ID id);
    boolean existsById(ID id);
    Iterable<T> findAll();
    long count();
    void deleteById(ID id);
    void delete(T entity);
}
```

12.3 JpaRepository (Recommended)

```java
// Extends CrudRepository + adds JPA features
public interface JpaRepository<T, ID> extends
        ListCrudRepository<T, ID>,
        ListPagingAndSortingRepository<T, ID>,
        QueryByExampleExecutor<T> {

    void flush();
```

```
    <S extends T> S saveAndFlush(S entity);
    void deleteAllInBatch();
    List<T> findAll(Sort sort);
    Page<T> findAll(Pageable pageable);
}

// Usage:
public interface ProductRepository extends JpaRepository<Product, Long> {
    // Inherits: save, findById, findAll, count, delete,
    //           flush, saveAndFlush, pagination, sorting, etc.
}
```

12.4 When to Use Which

Repository	Use When
`CrudRepository`	Simple CRUD, no pagination needed
`PagingAndSortingRepository`	Need pagination but no JPA features
`JpaRepository`	Default choice for Spring Boot apps

Interview Questions for Part 12

Q: What is the difference between CrudRepository and JpaRepository? A: JpaRepository extends CrudRepository and adds JPA-specific methods: `flush()`, `saveAndFlush()`, `deleteAllInBatch()`, and full pagination support. Always use JpaRepository in Spring Boot.

Q: Why does deleteAllInBatch() exist alongside deleteAll()? A: `deleteAll()` loads all entities first, then deletes each one individually (N queries). `deleteAllInBatch()` executes a single `DELETE FROM table` SQL (1 query). For large tables, batch is orders of magnitude faster.

Part 13: Query Methods in Spring Data JPA

Three Ways to Define Queries in Spring Data JPA

Spring Data JPA provides three strategies for creating queries, from simplest to most powerful:

Strategy	When to Use	Example
Derived query (method name)	Simple conditions (1-3 fields)	`findByName(String name)`
@Query (JPQL)	Complex queries, joins, aggregations	`@Query("SELECT u FROM User u WHERE...")`
Native query	Database-specific features, complex analytics	`@Query(value = "SELECT...", nativeQuery = true)`

> ***Rule of Thumb:*** *Start with derived queries. Move to @Query when method names become unreadable (more than 3 conditions). Use native queries only when you need database-specific syntax.*

Deep Theory: How Spring Data Derives Queries From Method Names

At startup, Spring Data parses each method name into tokens:

```
findByStatusAndRoleOrderByNameAsc
  |       |     |      |      |
  v       v     v      v      v
 find   Status  And   Role   OrderBy Name Asc
  |       |           |      |
  v       v           v      v
SELECT  WHERE status= AND role=  ORDER BY name ASC

Generated JPQL:
SELECT u FROM User u WHERE u.status = ?1 AND u.role = ?2 ORDER BY u.name ASC
```

If Spring Data cannot parse the method name, the application fails to start (fail-fast). This is actually a feature -- it catches query typos at startup rather than at runtime.

13.1 Method Name Query Derivation

How It Works

Spring Data parses method names and generates JPQL automatically. The method name IS the query -- Spring breaks it into parts:

```
public interface UserRepository extends JpaRepository<User, Long> {

    // SELECT u FROM User u WHERE u.name = ?1
    List<User> findByName(String name);

    // SELECT u FROM User u WHERE u.email = ?1
    Optional<User> findByEmail(String email);

    // SELECT u FROM User u WHERE u.status = ?1 AND u.role = ?2
    List<User> findByStatusAndRole(UserStatus status, Role role);

    // SELECT u FROM User u WHERE u.name LIKE %?1%
    List<User> findByNameContaining(String keyword);

    // SELECT u FROM User u WHERE u.age BETWEEN ?1 AND ?2
    List<User> findByAgeBetween(int min, int max);

    // SELECT u FROM User u WHERE u.status IN ?1
    List<User> findByStatusIn(Collection<UserStatus> statuses);

    // SELECT u FROM User u WHERE u.deletedAt IS NULL
    List<User> findByDeletedAtIsNull();

    // SELECT u FROM User u ORDER BY u.createdAt DESC
    List<User> findAllByOrderByCreatedAtDesc();

    // COUNT
    long countByStatus(UserStatus status);

    // EXISTS
    boolean existsByEmail(String email);

    // DELETE
    void deleteByStatus(UserStatus status);

    // LIMIT
    List<User> findTop10ByOrderByCreatedAtDesc();
    Optional<User> findFirstByOrderByCreatedAtDesc();
}
```

13.2 Query Keywords Reference

Keyword	SQL Equivalent	Example
`And`	AND	`findByNameAndEmail`
`Or`	OR	`findByNameOrEmail`
`Between`	BETWEEN	`findByAgeBetween`
`LessThan`	<	`findByAgeLessThan`
`GreaterThan`	>	`findByAgeGreaterThan`
`Like`	LIKE	`findByNameLike`
`Containing`	LIKE %...%	`findByNameContaining`
`StartingWith`	LIKE ...%	`findByNameStartingWith`
`In`	IN (...)	`findByStatusIn`
`IsNull`	IS NULL	`findByDeletedAtIsNull`
`IsNotNull`	IS NOT NULL	`findByEmailIsNotNull`
`OrderBy`	ORDER BY	`findByStatusOrderByNameAsc`
`Not`	<>	`findByStatusNot`
`True` / `False`	= true/false	`findByActiveTrue`

Part 14: Custom Queries

Deep Theory: Why @Query Exists

Derived query methods work great for 1-3 conditions. But when you need JOINs, GROUP BY, subqueries, or when the method name becomes `findByStatusAndRoleAndDepartmentAndSalaryGreaterThanAndCreatedAtAfter`, it's time for `@Query`. The query is validated at startup -- if you have a JPQL syntax error, the application fails to start.

14.1 @Query with JPQL

```
public interface OrderRepository extends JpaRepository<Order, Long> {

    // JPQL -- uses entity names, not table names
    @Query("SELECT o FROM Order o WHERE o.customer.id = :customerId " +
           "AND o.status = :status ORDER BY o.createdAt DESC")
    List<Order> findCustomerOrdersByStatus(
        @Param("customerId") Long customerId,
        @Param("status") OrderStatus status);

    // Projection -- return DTO instead of entity
    @Query("SELECT NEW com.app.dto.OrderSummary(o.id, o.totalAmount, " +
           "o.status, o.createdAt) FROM Order o " +
           "WHERE o.customer.id = :customerId")
    List<OrderSummary> findOrderSummaries(@Param("customerId") Long
customerId);

    // Aggregation
    @Query("SELECT o.status, COUNT(o), SUM(o.totalAmount) " +
           "FROM Order o GROUP BY o.status")
    List<Object[]> getOrderStatistics();

    // Update query
    @Modifying
    @Transactional
    @Query("UPDATE Order o SET o.status = :newStatus " +
           "WHERE o.status = :oldStatus AND o.createdAt < :cutoff")
    int bulkUpdateStatus(@Param("oldStatus") OrderStatus oldStatus,
                         @Param("newStatus") OrderStatus newStatus,
```

```
                                @Param("cutoff") LocalDateTime cutoff);
}
```

Production Warning: @Modifying Requires clearAutomatically

When you use `@Modifying` with `@Query` for UPDATE/DELETE, the persistence context is NOT automatically cleared. Entities loaded before the bulk operation still show old values!

```
@Modifying(clearAutomatically = true) // CRITICAL: clear stale cache
@Transactional
@Query("UPDATE Order o SET o.status = :newStatus WHERE ...")
int bulkUpdateStatus(...);
```

14.2 Native Queries

```
// When you need database-specific features
@Query(value = "SELECT * FROM orders WHERE total_amount > :min " +
               "ORDER BY created_at DESC LIMIT :limit",
       nativeQuery = true)
List<Order> findExpensiveRecentOrders(@Param("min") BigDecimal min,
                                      @Param("limit") int limit);

// Complex analytics
@Query(value = """
    SELECT DATE(o.created_at) as order_date,
           COUNT(*) as total_orders,
           SUM(o.total_amount) as revenue
    FROM orders o
    WHERE o.created_at >= :startDate
    GROUP BY DATE(o.created_at)
    ORDER BY order_date
    """, nativeQuery = true)
List<Object[]> getDailyRevenue(@Param("startDate") LocalDate startDate);
```

Part 15: Pagination and Sorting

Deep Theory: Why Pagination is Non-Negotiable in Production

Without pagination, `findAll()` on a table with 1 million rows loads ALL rows into memory -> OutOfMemoryError, or at minimum, a 30-second response time. Every list endpoint in production MUST be paginated.

Pagination generates TWO SQL queries:

1. `SELECT ... LIMIT 20 OFFSET 40` (fetch the page data)
2. `SELECT COUNT(*) FROM ...` (total count for pagination metadata)

The COUNT query can be expensive on large tables. For infinite-scroll UIs, use `Slice<T>` instead of `Page<T>` to skip the count query.

15.1 Pageable

```
// Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    Page<Product> findByCategory(String category, Pageable pageable);
}

// Service
@Transactional(readOnly = true)
public Page<ProductDTO> getProducts(String category, int page, int size) {
    Pageable pageable = PageRequest.of(page, size,
Sort.by("name").ascending());
    Page<Product> products = productRepository.findByCategory(category,
pageable);
    return products.map(this::toDTO); // Map entity to DTO
}

// Controller
@GetMapping("/api/products")
public Page<ProductDTO> listProducts(
        @RequestParam(defaultValue = "") String category,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size,
        @RequestParam(defaultValue = "name,asc") String[] sort) {

    Sort sortObj = Sort.by(Sort.Direction.fromString(sort[1]), sort[0]);
```

```
    return productService.getProducts(category, page, size);
}
```

15.2 Page Response

```
{
  "content": [ ... ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": { "sorted": true, "orders": [{"property": "name", "direction":
"ASC"}] }
  },
  "totalElements": 156,
  "totalPages": 8,
  "last": false,
  "first": true,
  "numberOfElements": 20
}
```

15.3 Sorting

```
// Multiple sort criteria
Sort sort = Sort.by(
    Sort.Order.desc("priority"),
    Sort.Order.asc("name")
);
List<Product> sorted = productRepository.findAll(sort);
```

Part 16: Spring Data JPA Specifications

What are Specifications?

Specifications allow you to build **dynamic, composable queries** at runtime. Instead of writing separate repository methods for every possible filter combination, you create reusable building blocks that can be combined with AND/OR.

Why Specifications Over Multiple Repository Methods

Imagine a product search page with 6 optional filters (category, price range, keyword, brand, rating, availability). Without Specifications, you'd need $2^6 = 64$ repository methods for every possible combination. With Specifications, you need just 6 reusable building blocks.

When to Use Specifications

- **Search/filter pages** with multiple optional criteria
- **Admin dashboards** where users combine filters dynamically
- When query conditions are known only at runtime

When NOT to Use Specifications

- Simple, fixed queries -- use derived query methods instead
- Complex analytics -- use native queries or views
- When there are only 1-2 filter conditions

Design Pattern: Specification Pattern (Domain-Driven Design)

This implements the **Specification pattern** from DDD -- each specification encapsulates a single business rule that can be combined with `and()`, `or()`, `not()`.

16.1 Dynamic Queries with Specifications

```
// Repository must extend JpaSpecificationExecutor
public interface ProductRepository extends JpaRepository<Product, Long>,
        JpaSpecificationExecutor<Product> {
}

// Specification class
public class ProductSpecifications {

    public static Specification<Product> hasCategory(String category) {
        return (root, query, cb) ->
```

```
            category == null ? null :
            cb.equal(root.get("category"), category);
    }

    public static Specification<Product> priceBetween(
            BigDecimal min, BigDecimal max) {
        return (root, query, cb) -> {
            if (min == null && max == null) return null;
            if (min != null && max != null)
                return cb.between(root.get("price"), min, max);
            if (min != null)
                return cb.greaterThanOrEqualTo(root.get("price"), min);
            return cb.lessThanOrEqualTo(root.get("price"), max);
        };
    }

    public static Specification<Product> nameContains(String keyword) {
        return (root, query, cb) ->
            keyword == null ? null :
            cb.like(cb.lower(root.get("name")),
                    "%" + keyword.toLowerCase() + "%");
    }

    public static Specification<Product> isActive() {
        return (root, query, cb) ->
            cb.equal(root.get("active"), true);
    }
}

// Service -- compose specifications dynamically
@Transactional(readOnly = true)
public Page<Product> searchProducts(ProductSearchRequest req, Pageable
pageable) {
    Specification<Product> spec = Specification
        .where(ProductSpecifications.isActive())
        .and(ProductSpecifications.hasCategory(req.getCategory()))
        .and(ProductSpecifications.priceBetween(req.getMinPrice(),
req.getMaxPrice()))
        .and(ProductSpecifications.nameContains(req.getKeyword()));

    return productRepository.findAll(spec, pageable);
}
```

Performance Tip: Avoid N+1 with Specifications

Specifications generate JPQL that can trigger N+1 if the matched entities have LAZY associations that you access later. Combine Specifications with `@EntityGraph` or a custom `Specification` that adds a `JOIN FETCH`:

```
public static Specification<Product> fetchCategory() {
    return (root, query, cb) -> {
        if (query.getResultType() != Long.class) { // Skip for COUNT queries
            root.fetch("category", JoinType.LEFT);
        }
        return null;
    };
}
```

Interview Questions for Part 16

Q: When should you use Specifications vs derived query methods? A: Use Specifications when you have dynamic, optional filters that combine at runtime (search pages with multiple checkboxes). Use derived methods for fixed, simple queries.

Q: Can you combine Specifications with pagination? A: Yes. Pass both `Specification<T>` and `Pageable` to `repository.findAll(spec, pageable)`. The Specification handles WHERE clauses, Pageable handles LIMIT/OFFSET/ORDER BY.

Part 17: Performance Optimization

Deep Theory: The Three Pillars of Hibernate Performance

Every Hibernate performance problem falls into one of three categories:

1. **Too many queries** (N+1 problem) -- Fix with JOIN FETCH, @EntityGraph, @BatchSize
2. **Too much data** (loading full entities when only 3 columns needed) -- Fix with DTO projections
3. **Too long transactions** (holding DB connection for seconds) -- Fix by separating external calls

Mastering these three areas eliminates 95% of Hibernate performance issues.

17.1 Batch Fetching

```
// application.yml
spring:
  jpa:
    properties:
      hibernate:
        default_batch_fetch_size: 25
        jdbc:
          batch_size: 50
        order_inserts: true
        order_updates: true
```

```
// Entity-level batch size
@Entity
public class Customer {
    @OneToMany(mappedBy = "customer")
    @BatchSize(size = 25) // Fetch orders in batches of 25
    private List<Order> orders;
}
// Instead of N queries, generates: SELECT * FROM orders WHERE customer_id IN
(?,?,?,...,?)
```

17.2 Connection Pooling (HikariCP)

Deep Theory: Why Connection Pool Sizing Matters

Opening a database connection takes 20-50ms (TCP handshake, SSL negotiation, authentication). HikariCP pre-creates connections so they're ready instantly.

Formula for pool size: `connections = (core_count * 2) + effective_spindle_count` For a 4-core server with SSD: `(4 * 2) + 1 = 9` connections max. Most apps work well with 10-20.

Warning: Setting pool size too high HURTS performance. With 100 connections, the database spends more time context-switching between connections than executing queries.

```yaml
spring:
  datasource:
    hikari:
      maximum-pool-size: 20        # Max connections
      minimum-idle: 5              # Min idle connections
      idle-timeout: 300000         # 5 min idle timeout
      connection-timeout: 20000    # 20 sec connection timeout
      max-lifetime: 1800000        # 30 min max connection lifetime
      leak-detection-threshold: 60000  # Detect connection leaks
```

Production Tip -- leak-detection-threshold: If a connection is checked out for longer than this threshold (60s), HikariCP logs a warning with the stack trace showing WHERE the connection was borrowed. This is invaluable for finding long-running transactions.

17.3 DTO Projections (Avoid Fetching Full Entities)

Deep Theory: Why DTO Projections are Critical

Loading a full entity triggers:

1. ALL columns selected (including large TEXT/BLOB)
2. Entity snapshot created (400 bytes per entity for dirty checking)
3. Entity stored in persistence context (memory)
4. All EAGER associations loaded (more queries)

A DTO projection avoids ALL of this overhead.

Projection Type	Performance	Type Safety	Use When
Interface-based	Good	Compile-time	Simple projections
Class-based (record)	Best	Compile-time	Complex projections

Tuple / Object[]	Best	No	Quick analytics

```
// Interface-based projection
public interface UserSummary {
    Long getId();
    String getName();
    String getEmail();
}

public interface UserRepository extends JpaRepository<User, Long> {
    List<UserSummary> findByStatus(UserStatus status);
    // Generates: SELECT u.id, u.name, u.email FROM users u WHERE u.status = ?
    // NOT: SELECT * FROM users ... (avoids loading unnecessary columns)
}

// Class-based projection (DTO) -- best performance
public record UserDTO(Long id, String name, String email) {}

@Query("SELECT new com.app.dto.UserDTO(u.id, u.name, u.email) " +
       "FROM User u WHERE u.status = :status")
List<UserDTO> findUserDTOs(@Param("status") UserStatus status);
```

17.4 Bulk Operations

```
// BAD: Loading and saving one by one
for (Long id : userIds) {
    User user = userRepository.findById(id).get();
    user.setStatus(UserStatus.INACTIVE);
    userRepository.save(user); // N SELECT + N UPDATE queries!
}

// GOOD: Bulk update with @Query
@Modifying
@Transactional
@Query("UPDATE User u SET u.status = :status WHERE u.id IN :ids")
int bulkUpdateStatus(@Param("ids") List<Long> ids,
                     @Param("status") UserStatus status);
// Single UPDATE query regardless of list size
```

Interview Questions for Part 17

Q: How do you decide between interface-based and class-based DTO projections? A: Interface-based projections are simpler to write but slightly slower (Spring creates proxies). Class-based (records) are fastest because Hibernate creates objects directly. Use records for performance-critical paths.

Q: What is the formula for HikariCP pool size? A: `(core_count * 2) + spindle_count`. For a 4-core server with SSD: ~9 connections. Setting pool size too high causes database context-switching overhead and actually degrades performance.

Part 18: Hibernate Caching

Deep Theory: Cache Lookup Flow

When you call `entityManager.find(User.class, 1L)`, Hibernate searches caches in order:

```
find(User.class, 1L)
  |
  v
1st Level Cache (Persistence Context) -- ALWAYS checked first
  |-- FOUND? Return immediately (same object reference)
  |-- NOT FOUND?
  v
2nd Level Cache (if enabled) -- Shared across sessions
  |-- FOUND? Hydrate entity from cached data, add to 1st level cache
  |-- NOT FOUND?
  v
Database Query -- SELECT * FROM users WHERE id = 1
  |-- Result mapped to entity
  |-- Stored in 1st level cache
  |-- Stored in 2nd level cache (if enabled for this entity)
  |-- Returned to caller
```

18.1 First-Level Cache (Session Cache)

What: Built-in, always-on cache within each Hibernate Session (EntityManager).

Scope: Per session (per transaction in Spring).

How it works:

```java
@Transactional
public void demonstrateFirstLevelCache() {
    // Query 1: Hits database
    User u1 = userRepository.findById(1L).get();
    // SQL: SELECT * FROM users WHERE id = 1

    // Query 2: Returns from cache -- NO database hit!
    User u2 = userRepository.findById(1L).get();
    // No SQL executed!
```

```
    assert u1 == u2; // Same object reference (identity guarantee)
}
```

18.2 Second-Level Cache

What: Shared cache across all Sessions. Optional, must be configured.

Scope: SessionFactory-level (application-wide).

```
+---------------------------------+
|     Application (Multiple       |
|     Concurrent Requests)        |
+---------------------------------+
   |          |           |
   v          v           v
+------+  +------+    +------+
|Sess 1|  |Sess 2|    |Sess 3|  <-- Each has own 1st level cache
+------+  +------+    +------+
     \       |        /
      +------+-------+
      | 2nd Level     |  <-- Shared across ALL sessions
      | Cache         |
      | (EhCache/     |
      |  Redis/       |
      |  Hazelcast)   |
      +-------+-------+
              |
              v
      +-------+-------+
      |   Database     |
      +---------------+
```

Configuration

```
<!-- pom.xml -->
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jcache</artifactId>
</dependency>
<dependency>
    <groupId>org.ehcache</groupId>
```

```
    <artifactId>ehcache</artifactId>
</dependency>
```

```yaml
# application.yml
spring:
  jpa:
    properties:
      hibernate:
        cache:
          use_second_level_cache: true
          region.factory_class: jcache
        javax:
          cache:
            provider: org.ehcache.jsr107.EhcacheCachingProvider
```

```java
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private BigDecimal price;
}
// Now Product entities are cached in 2nd level cache
// Subsequent findById() calls return from cache, not DB
```

Cache Strategies

Strategy	Description	Use When
`READ_ONLY`	Never changes after creation	Reference data, countries, currencies
`READ_WRITE`	Reads and updates	Most entities
`NONSTRICT_READ_WRITE`	Eventual consistency	Data where slight staleness is OK
`TRANSACTIONAL`	Full transaction support	JTA environments

18.3 Query Cache

```yaml
spring:
  jpa:
    properties:
      hibernate:
        cache:
          use_query_cache: true
```

```java
@QueryHints(@QueryHint(name = "org.hibernate.cacheable", value = "true"))
List<Product> findByCategory(String category);
// Results of this query are cached
// Invalidated when any Product entity changes
```

Production Warning: Query Cache Invalidation Trap

The query cache invalidates ALL cached queries for an entity type whenever ANY entity of that type is inserted, updated, or deleted. If you have a frequently-updated `Product` table, caching `findByCategory` is pointless -- the cache is invalidated on every product update.

Use query cache ONLY for: Rarely-changed reference data (countries, currencies, configuration values).

Interview Questions for Part 18

Q: What is the difference between 1st and 2nd level cache? A: 1st level cache is per-session, always-on, and guarantees object identity (same ID = same reference). 2nd level cache is shared across all sessions, optional, and must be explicitly configured per entity.

Q: When should you NOT use the query cache? A: When the underlying entity table is frequently updated. The query cache invalidates ALL cached queries for an entity type on any INSERT/UPDATE/DELETE, making it counterproductive for actively modified tables.

Part 19: Common Performance Problems

19.1 N+1 Query Problem

Problem: Loading a list of entities, then lazy-loading a relationship for each one.

```
// This generates 1 + N queries:
List<Customer> customers = customerRepository.findAll(); // 1 query
for (Customer c : customers) {
    c.getOrders().size(); // N queries (one per customer)
}
```

Solutions:

Solution	Method	Query Count
JOIN FETCH	`@Query("SELECT c FROM Customer c JOIN FETCH c.orders")`	1
@EntityGraph	`@EntityGraph(attributePaths = {"orders"})`	1
@BatchSize	`@BatchSize(size = 25)` on collection	N/25
DTO Projection	Select only needed columns	1

19.2 Lazy Loading Exceptions

Problem: `LazyInitializationException` when accessing lazy-loaded data outside Session.

```
// BAD: Session is closed when controller tries to access orders
@GetMapping("/customers/{id}")
public Customer getCustomer(@PathVariable Long id) {
    Customer c = customerService.findById(id); // Session closes after service
    c.getOrders().size(); // LazyInitializationException!
}
```

Solutions:

```java
// Solution 1: Fetch eagerly in the query (BEST)
@Query("SELECT c FROM Customer c JOIN FETCH c.orders WHERE c.id = :id")
Customer findByIdWithOrders(@Param("id") Long id);

// Solution 2: @Transactional on the service method
@Transactional(readOnly = true)
public CustomerDTO getCustomerWithOrders(Long id) {
    Customer c = customerRepository.findById(id).orElseThrow();
    return new CustomerDTO(c, c.getOrders()); // Access while session is open
}

// Solution 3: EntityGraph
@EntityGraph(attributePaths = {"orders"})
Optional<Customer> findById(Long id);
```

Production Warning: NEVER Use Open Session in View

`spring.jpa.open-in-view=true` (Spring Boot default!) keeps the Hibernate Session open through the entire HTTP request, including view rendering. This prevents LazyInitializationException but:

- Holds database connections for the entire request duration
- Leads to N+1 queries in controllers/templates
- Causes connection pool exhaustion under load

Always disable it: `spring.jpa.open-in-view=false`

19.3 Slow Queries

Diagnosis:

```yaml
spring:
  jpa:
    properties:
      hibernate:
        generate_statistics: true  # Log query statistics
        format_sql: true
    show-sql: true

logging:
  level:
```

```
    org.hibernate.SQL: DEBUG
    org.hibernate.orm.jdbc.bind: TRACE  # Show parameter values
```

Solutions:

- Add database indexes for frequently queried columns
- Use DTO projections instead of full entities
- Add `@BatchSize` for collections
- Use pagination for large result sets
- Avoid `SELECT *` via projections

19.4 Memory Issues

```
// BAD: Loading 1 million entities into memory
List<User> all = userRepository.findAll(); // OutOfMemoryError!

// GOOD: Process in pages
int page = 0;
Page<User> batch;
do {
    batch = userRepository.findAll(PageRequest.of(page, 500));
    batch.getContent().forEach(this::processUser);
    entityManager.clear(); // Clear persistence context to free memory
    page++;
} while (batch.hasNext());
```

Interview Questions for Part 19

Q: What causes LazyInitializationException? A: Accessing a LAZY association after the Hibernate Session/EntityManager has been closed. The proxy can't execute the SQL to load data. Fix: use JOIN FETCH, @EntityGraph, or ensure access happens within a @Transactional method.

Q: Should you enable spring.jpa.open-in-view? A: No. It holds database connections for the entire HTTP request including view rendering, leading to connection pool exhaustion. Always disable it and use JOIN FETCH or @EntityGraph instead.

Part 20: Real Production Scenario

20.1 User Management System

Architecture

```
+-----------+     +-------------+     +--------------+     +----------+
|  Client   | --> | Controller  | --> |   Service    | --> |Repository|
| (React)   |     | (REST API)  |     | (@Transact.) |     | (JPA)    |
+-----------+     +-------------+     +--------------+     +----------+
                                                                 |
                                                                 v
                                                            +----------+
                                                            | Database |
                                                            |PostgreSQL|
                                                            +----------+
```

Entity

```java
@Entity
@Table(name = "users",
    indexes = @Index(name = "idx_user_email", columnList = "email"))
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE,
        generator = "user_seq")
    @SequenceGenerator(name = "user_seq",
        sequenceName = "user_sequence", allocationSize = 50)
    private Long id;

    @Column(nullable = false, length = 100)
    private String firstName;

    @Column(nullable = false, length = 100)
    private String lastName;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
```

```java
    private String passwordHash;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private UserStatus status = UserStatus.ACTIVE;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role = Role.USER;

    @Embedded
    private Address address;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<UserSession> sessions = new ArrayList<>();

    @Column(updatable = false)
    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = createdAt;
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

Repository

```java
public interface UserRepository extends JpaRepository<User, Long>,
        JpaSpecificationExecutor<User> {

    Optional<User> findByEmail(String email);

    boolean existsByEmail(String email);

    @Query("SELECT u FROM User u WHERE u.status = :status")
    Page<User> findByStatus(@Param("status") UserStatus status,
```

```
                                Pageable pageable);

    @EntityGraph(attributePaths = {"sessions"})
    Optional<User> findWithSessionsById(Long id);

    @Modifying
    @Query("UPDATE User u SET u.status = 'INACTIVE' " +
           "WHERE u.updatedAt < :cutoff AND u.status = 'ACTIVE'")
    int deactivateInactiveUsers(@Param("cutoff") LocalDateTime cutoff);

    @Query("SELECT new com.app.dto.UserSummaryDTO(u.id, u.firstName, " +
           "u.lastName, u.email, u.status) FROM User u")
    Page<UserSummaryDTO> findAllSummaries(Pageable pageable);
}
```

Service

```
@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Transactional
    public UserDTO createUser(CreateUserRequest request) {
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new DuplicateEmailException(request.getEmail());
        }

        User user = new User();
        user.setFirstName(request.getFirstName());
        user.setLastName(request.getLastName());
        user.setEmail(request.getEmail());
        user.setPasswordHash(passwordEncoder.encode(request.getPassword()));
        user.setRole(request.getRole());

        User saved = userRepository.save(user);
        return UserDTO.from(saved);
    }

    @Transactional(readOnly = true)
    public Page<UserSummaryDTO> listUsers(int page, int size) {
        return userRepository.findAllSummaries(
            PageRequest.of(page, size, Sort.by("createdAt").descending()));
```

```
    }

    @Transactional
    public UserDTO updateUser(Long id, UpdateUserRequest request) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
        user.setFirstName(request.getFirstName());
        user.setLastName(request.getLastName());
        // Dirty checking auto-saves changes on commit
        return UserDTO.from(user);
    }
}
```

Controller

```
@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @PostMapping
    public ResponseEntity<UserDTO> createUser(@Valid @RequestBody
CreateUserRequest req) {
        UserDTO user = userService.createUser(req);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }

    @GetMapping
    public Page<UserSummaryDTO> listUsers(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size) {
        return userService.listUsers(page, size);
    }

    @PutMapping("/{id}")
    public UserDTO updateUser(@PathVariable Long id,
                              @Valid @RequestBody UpdateUserRequest req) {
        return userService.updateUser(id, req);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
```

```
        return ResponseEntity.noContent().build();
    }
}
```

Part 21: Microservices Data Strategy

21.1 Database Per Service Pattern

```
+------------------+    +------------------+    +------------------+
|  User Service    |    |  Order Service   |    | Payment Service  |
|  +-----------+   |    |  +-----------+   |    |  +-----------+   |
|  | Spring    |   |    |  | Spring    |   |    |  | Spring    |   |
|  | Data JPA  |   |    |  | Data JPA  |   |    |  | Data JPA  |   |
|  +-----------+   |    |  +-----------+   |    |  +-----------+   |
|       |          |    |       |          |    |       |          |
|  +-----------+   |    |  +-----------+   |    |  +-----------+   |
|  | User DB   |   |    |  | Order DB  |   |    |  | Payment DB|   |
|  | PostgreSQL|   |    |  | MySQL     |   |    |  | PostgreSQL|   |
|  +-----------+   |    |  +-----------+   |    |  +-----------+   |
+------------------+    +------------------+    +------------------+
```

Key Rules:

1. Each service owns its database -- no direct cross-service DB access
2. Services communicate via APIs or events, never via shared DB
3. Each service has its own `application.yml` with its own datasource
4. **NO @ManyToOne or @OneToMany across service boundaries** -- use IDs and API calls

Deep Theory: Why Database Per Service

Sharing a database between services creates **tight coupling**:

- Schema changes in one service break another
- One service's slow query blocks another's connections
- You can't independently scale or deploy services
- Transaction boundaries become unclear

21.2 Saga Pattern (Distributed Transactions)

```
Order Saga (Place Order):

1. Order Service: Create order (PENDING)
         |
```

```
         v
2. Payment Service: Reserve payment
         |
    Success? ----No----> Compensate: Cancel order
         |
        Yes
         |
         v
3. Inventory Service: Reserve stock
         |
    Success? ----No----> Compensate: Refund payment, Cancel order
         |
        Yes
         |
         v
4. Order Service: Confirm order (CONFIRMED)
```

```java
// Orchestration-based Saga
@Service
public class OrderSagaOrchestrator {

    @Transactional
    public OrderResult placeOrder(OrderRequest request) {
        // Step 1: Create order
        Order order = orderService.create(request);

        try {
            // Step 2: Reserve payment
            PaymentResult payment = paymentClient.reserve(
                order.getId(), order.getTotalAmount());

            // Step 3: Reserve inventory
            InventoryResult inventory = inventoryClient.reserve(
                order.getItems());

            // Step 4: Confirm
            order.confirm();
            return OrderResult.success(order);

        } catch (PaymentException e) {
            order.cancel("Payment failed");
            throw e;
        } catch (InventoryException e) {
            paymentClient.refund(order.getId()); // Compensate
            order.cancel("Inventory unavailable");
            throw e;
```

```
        }
    }
}
```

21.3 Eventual Consistency with Events

```
// Order Service publishes event after saving
@Transactional
public Order createOrder(OrderRequest request) {
    Order order = orderRepository.save(new Order(request));

    // Publish event via Kafka
    kafkaTemplate.send("order-events",
        new OrderCreatedEvent(order.getId(), order.getCustomerId(),
            order.getTotalAmount()));

    return order;
}

// Inventory Service consumes event
@KafkaListener(topics = "order-events")
public void handleOrderCreated(OrderCreatedEvent event) {
    inventoryService.reserveStock(event.getOrderId(), event.getItems());
}
```

Deep Theory: The Outbox Pattern (Reliable Events)

The code above has a critical flaw: if `orderRepository.save()` succeeds but `kafkaTemplate.send()` fails, you have an order in the DB but no event published. The **Outbox Pattern** fixes this:

```
@Transactional
public Order createOrder(OrderRequest request) {
    Order order = orderRepository.save(new Order(request));
    // Save event to outbox table IN THE SAME TRANSACTION
    outboxRepository.save(new OutboxEvent("OrderCreated", order.getId()));
    return order;
    // A separate poller reads outbox table and publishes to Kafka
```

```
    // This guarantees: if order is saved, event is also saved (same TX)
}
```

Interview Questions for Part 21

Q: Why can't you use @ManyToOne across microservices? A: Because each service has its own database. JPA relationships require both tables to be in the same database. Cross-service data access uses REST APIs or events.

Q: What is the Saga pattern? A: A sequence of local transactions across services, where each step either succeeds or triggers compensating actions to undo previous steps. Used instead of distributed transactions in microservices.

Part 22: Build It Yourself -- User Management with JPA

> ***Goal:*** *Build a Spring Boot User Management REST API using Hibernate/JPA, covering entities, repositories, service layer, and performance optimization.*

Concept Overview

Layer	Technology	Purpose
Entity	`@Entity`, `@Table`, `@ManyToOne`	Maps Java objects to database tables
Repository	`JpaRepository`, `@Query`	Data access without writing SQL
Service	`@Transactional`, dirty checking	Business logic with automatic transaction management
DTO	Record classes	Decouple API from internal entities

Step 1: Define Entities with Relationships

Concept: Each `@Entity` class maps to a database table. Relationships use `@ManyToOne` (owning side with FK) and `@OneToMany` (inverse side).

```
@Entity
@Table(name = "users",
    indexes = @Index(name = "idx_user_email", columnList = "email"))
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String firstName;

    @Column(nullable = false, unique = true)
    private String email;

    @Enumerated(EnumType.STRING)
```

```
    private UserStatus status = UserStatus.ACTIVE;

    @Embedded                                   // Value object stored in same
table
    private Address address;

    @OneToMany(mappedBy = "user",               // Inverse side -- no FK here
               cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Order> orders = new ArrayList<>();

    @Column(updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    void onCreate() { createdAt = LocalDateTime.now(); }

    // Helper method for bidirectional consistency
    public void addOrder(Order order) {
        orders.add(order);
        order.setUser(this);
    }
}
```

Step 2: Create the Repository

Concept: `JpaRepository` gives you CRUD + paging for free. Custom queries use `@Query` (JPQL) or derived method names.

```
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);          // Derived query --
Spring generates SQL

    boolean existsByEmail(String email);               // EXISTS query --
efficient

    @Query("SELECT u FROM User u WHERE u.status = :status")
    Page<User> findByStatus(@Param("status") UserStatus status, Pageable
pageable);

    @EntityGraph(attributePaths = {"orders"})           // Fetch orders eagerly
(avoids N+1)
```

```
    Optional<User> findWithOrdersById(Long id);

    @Modifying @Transactional
    @Query("UPDATE User u SET u.status = 'INACTIVE' WHERE u.createdAt <
:cutoff")
    int deactivateOldUsers(@Param("cutoff") LocalDateTime cutoff);
}
```

Step 3: Build the Service Layer

Concept: `@Transactional` manages database transactions. Dirty checking automatically saves changes to persistent entities -- no explicit `save()` needed for updates.

```
@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;

    @Transactional
    public UserDTO createUser(CreateUserRequest req) {
        if (userRepository.existsByEmail(req.email())) {
            throw new DuplicateEmailException(req.email());
        }
        User user = User.builder()
            .firstName(req.firstName())
            .email(req.email())
            .build();
        return UserDTO.from(userRepository.save(user));
    }

    @Transactional(readOnly = true)       // Read-only = no dirty checking
overhead
    public Page<UserDTO> listUsers(int page, int size) {
        return userRepository.findAll(PageRequest.of(page, size))
            .map(UserDTO::from);
    }

    @Transactional
    public UserDTO updateUser(Long id, UpdateUserRequest req) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
        user.setFirstName(req.firstName());
```

```
        // NO save() needed -- dirty checking auto-persists on commit
        return UserDTO.from(user);
    }
}
```

Step 4: Performance Optimization

```yaml
spring:
  jpa:
    properties:
      hibernate:
        default_batch_fetch_size: 25     # Batch-fetch lazy collections
        jdbc:
          batch_size: 50                  # Batch INSERTs/UPDATEs
        order_inserts: true               # Group INSERTs by entity type
        order_updates: true
    show-sql: true                        # See generated SQL
```

Key Takeaways

Concept	Remember
`@ManyToOne(fetch = LAZY)`	Always set LAZY -- override EAGER defaults
`mappedBy`	Inverse side has NO FK column -- it's a mirror
Dirty checking	Persistent entities auto-save on transaction commit
`@EntityGraph`	Solve N+1 by fetching associations in a single query
`@Transactional(readOnly = true)`	Better performance for read operations
DTO projections	Don't expose entities directly -- use DTOs

Part 22: Design Patterns Used

22.1 Repository Pattern

What: Encapsulates data access logic behind an interface, providing a collection-like API.

Why: Decouples business logic from data access. Enables testing with mock repositories.

```
// The Repository Pattern is built into Spring Data JPA
public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByCustomerId(Long customerId);
}

// Service depends on interface, not implementation
@Service
public class OrderService {
    private final OrderRepository orderRepository; // Interface!

    public List<Order> getCustomerOrders(Long customerId) {
        return orderRepository.findByCustomerId(customerId);
    }
}

// In tests: use mock
@MockBean
private OrderRepository orderRepository;
```

22.2 DAO Pattern (Data Access Object)

What: Similar to Repository but closer to the persistence mechanism. Often wraps EntityManager directly.

When: When Spring Data JPA's auto-generated methods are insufficient.

```
@Repository
public class CustomOrderDao {

    @PersistenceContext
    private EntityManager em;

    public List<Order> findByComplexCriteria(OrderSearchCriteria criteria) {
```

```
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Order> cq = cb.createQuery(Order.class);
        Root<Order> root = cq.from(Order.class);
        List<Predicate> predicates = new ArrayList<>();
        // Build dynamic query...
        return em.createQuery(cq).getResultList();
    }
}
```

22.3 Specification Pattern

What: Encapsulates query criteria into reusable, composable objects.

When: Dynamic filtering (search pages, dashboards).

```
// Already covered in Part 16 -- Specifications are composable:
Specification<Product> spec = Specification
    .where(hasCategory("Electronics"))
    .and(priceBetween(100, 500))
    .and(nameContains("phone"));
productRepository.findAll(spec, pageable);
```

22.4 Unit of Work Pattern

What: Tracks all changes during a business transaction and persists them as a single unit.

How Hibernate implements it: The Session/EntityManager IS the Unit of Work. It tracks all persistent entities and flushes changes on commit.

Internal Mechanism:

```
Unit of Work maintains:
  1. IdentityMap -- tracks all loaded entities by type + ID
  2. EntitySnapshots -- original state for dirty checking
  3. ActionQueue -- pending INSERT/UPDATE/DELETE actions
  4. CollectionEntries -- tracks collection modifications

On flush():
  1. Dirty check ALL entities (compare current vs snapshot)
  2. Sort actions (INSERTs first, DELETEs last for FK integrity)
  3. Generate SQL via Dialect
```

```
  4. Execute batch JDBC operations
  5. Update snapshots to current state
```

```java
@Transactional // Defines the Unit of Work boundary
public void processOrder(OrderRequest request) {
    // All changes within this method are tracked
    Order order = new Order(request);
    entityManager.persist(order);      // Tracked

    Customer customer = customerRepo.findById(request.getCustomerId()).get();
    customer.incrementOrderCount();    // Tracked (dirty checking)

    Inventory inv = inventoryRepo.findByProductId(request.getProductId());
    inv.decrementStock(request.getQuantity()); // Tracked

    // At @Transactional boundary: ALL changes flushed in single transaction
    // INSERT order + UPDATE customer + UPDATE inventory
}
```

22.5 Identity Map Pattern

What: Ensures that within a single Unit of Work (Session), each database row is represented by exactly ONE Java object. Loading User(id=1) twice returns the same object reference.

Why it matters: Without Identity Map, two different objects could represent the same row, leading to conflicting updates.

```java
@Transactional
public void demo() {
    User u1 = userRepo.findById(1L).get();
    User u2 = userRepo.findById(1L).get();

    assert u1 == u2; // TRUE! Same object reference (Identity Map)

    u1.setName("Alice");
    System.out.println(u2.getName()); // "Alice" -- same object!
}
```

Part 23: Best Practices

23.1 Entity Design

Deep Theory: Why equals/hashCode Must Use Business Key

Hibernate uses `equals()` and `hashCode()` for Set operations, merge detection, and second-level cache lookups. Using `@Id` for equals/hashCode is broken because:

- Before `persist()`, id is `null` -- entities can't be in Sets
- After `persist()`, id changes from null to a value -- hashCode changes, breaking HashMap contract

Rule: Use a stable business key (email, ISBN, SSN) or a UUID assigned in the constructor.

```
// 1. Always override equals() and hashCode() using business key
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;

    @Column(unique = true, nullable = false)
    private String email; // Business key

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof User)) return false;
        User user = (User) o;
        return email != null && email.equals(user.email);
    }

    @Override
    public int hashCode() {
        return Objects.hash(email); // Based on business key, NOT id
    }
}

// 2. Use LAZY fetch by default
@ManyToOne(fetch = FetchType.LAZY) // Override EAGER default!
private Customer customer;
```

```java
// 3. Use Set instead of List for ManyToMany (avoids delete-all + re-insert)
@ManyToMany
private Set<Role> roles = new HashSet<>();

// 4. Add helper methods for bidirectional relationships
public void addOrder(Order order) {
    orders.add(order);
    order.setCustomer(this);
}

// 5. Use @PrePersist and @PreUpdate for audit fields
@PrePersist void onCreate() { createdAt = Instant.now(); }
@PreUpdate void onUpdate() { updatedAt = Instant.now(); }
```

23.2 Transaction Boundaries

```java
// 1. Keep transactions SHORT -- only in service layer
@Service
public class UserService {
    @Transactional // HERE -- not in controller or repository
    public void updateUser(Long id, UpdateRequest req) { ... }
}

// 2. Use readOnly for queries (performance boost)
@Transactional(readOnly = true)
public List<UserDTO> searchUsers(SearchCriteria criteria) { ... }

// 3. Avoid calling external APIs inside transactions
@Transactional
public void processPayment(PaymentRequest req) {
    Order order = orderRepo.findById(req.getOrderId()).get();
    // DON'T call external payment API here -- holds DB connection!
}

// BETTER:
public void processPayment(PaymentRequest req) {
    Order order = loadOrder(req.getOrderId()); // @Transactional(readOnly=true)
    PaymentResult result = paymentApi.charge(req); // No transaction
    updateOrderStatus(order.getId(), result); // @Transactional
}
```

23.3 Avoiding LazyInitializationException

```java
// Rule: Access all lazy data WITHIN the @Transactional boundary

// Option 1: JOIN FETCH in repository
@Query("SELECT c FROM Customer c JOIN FETCH c.orders WHERE c.id = :id")
Customer findWithOrders(@Param("id") Long id);

// Option 2: @EntityGraph
@EntityGraph(attributePaths = {"orders", "orders.items"})
Optional<Customer> findById(Long id);

// Option 3: Map to DTO inside transaction
@Transactional(readOnly = true)
public CustomerDTO getCustomer(Long id) {
    Customer c = customerRepo.findById(id).orElseThrow();
    return new CustomerDTO(c.getName(), c.getOrders().size()); // Access here!
}
```

23.4 Query Writing Rules

1. **Never use** `SELECT *` **in production** -- use DTO projections
2. **Add indexes** for columns used in WHERE, JOIN, ORDER BY
3. **Paginate large results** -- never return unbounded lists
4. **Use @BatchSize** for collections to prevent N+1
5. **Use bulk operations** (@Modifying @Query) for mass updates
6. **Monitor slow queries** with `hibernate.generate_statistics: true`

Part 24: Interview Questions (100+)

Hibernate Fundamentals (Q1-Q25)

Q1. What is ORM and why do we need it? ORM maps Java objects to database tables automatically, eliminating manual JDBC code, ResultSet mapping, and SQL string building. It solves the object-relational impedance mismatch.

Q2. What is the difference between JPA and Hibernate? JPA is a specification (interface/standard). Hibernate is an implementation of JPA. JPA defines annotations like `@Entity`, `@Id`; Hibernate provides the actual code that makes them work.

Q3. What is a SessionFactory? A heavyweight, thread-safe, immutable factory created once at startup. It holds compiled mappings, caches, and connection pool configuration. One SessionFactory per database.

Q4. What is a Session/EntityManager? A lightweight, short-lived, non-thread-safe object that wraps a JDBC connection. It provides the API for CRUD operations and serves as the first-level cache.

Q5. Explain the Hibernate entity lifecycle. Four states: Transient (new, not managed), Persistent (managed by Session, changes auto-tracked), Detached (was persistent, Session closed), Removed (scheduled for deletion).

Q6. What is dirty checking? Hibernate automatically detects changes to persistent entities by comparing current state to a snapshot taken when the entity was loaded. Modified entities generate UPDATE SQL on flush.

Q7. What is the first-level cache? The Session-level cache. Within a Session, the same entity (by ID) is always the same Java object. Second `findById()` returns the cached instance, not a new DB query.

Q8. What is the second-level cache? A SessionFactory-level cache shared across all Sessions. Requires configuration (EhCache, Hazelcast). Caches entity data beyond individual Sessions.

Q9. What is the difference between `persist()` and `save()`? `persist()` is JPA standard -- makes entity persistent, may not return ID immediately. `save()` is Hibernate-specific -- returns the generated ID immediately.

Q10. What is the difference between `merge()` and `update()`? `merge()` copies detached entity state to a new persistent instance (returns the managed copy). `update()` reattaches the detached entity itself (Hibernate-specific, can throw if another instance with same ID is in Session).

Q11. What is `@GeneratedValue` strategy SEQUENCE vs IDENTITY? SEQUENCE uses a database sequence object, allowing batch inserts (Hibernate pre-allocates IDs). IDENTITY uses auto-increment columns but disables batch inserts because Hibernate must execute INSERT to get the ID.

Q12. What is an @Embeddable? A value object that maps to columns in the parent entity's table (no separate table). Example: Address embedded in Customer table.

Q13. What is the difference between @Embedded and @OneToOne? @Embedded stores data in the same table (value object, no identity). @OneToOne uses a separate table with foreign key (has its own identity/lifecycle).

Q14. What is cascade in Hibernate? Cascade propagates operations from parent to child. CascadeType.PERSIST cascades save, REMOVE cascades delete, ALL cascades everything.

Q15. What is orphanRemoval? When a child entity is removed from the parent's collection, it is automatically deleted from the database. Different from CASCADE.REMOVE which only deletes children when the parent is deleted.

Q16. Explain owning side vs inverse side. The owning side has the foreign key column (@JoinColumn). The inverse side uses `mappedBy`. Only changes to the owning side are persisted to the database.

Q17. What is FetchType.LAZY vs EAGER? LAZY loads associated data only when accessed (proxy). EAGER loads immediately via JOIN. LAZY is preferred for performance.

Q18. What is the N+1 problem? Loading N entities and then lazily loading a relationship for each generates 1 + N queries. Solved with JOIN FETCH, @EntityGraph, or @BatchSize.

Q19. What is HQL vs JPQL? HQL is Hibernate-specific query language. JPQL is the JPA standard. Both operate on entities (not tables). JPQL is portable across JPA providers.

Q20. When would you use native SQL? For database-specific features (window functions, CTEs, full-text search), complex queries that are difficult in JPQL, or performance-critical queries.

Q21. What is @NamedQuery? Pre-compiled JPQL queries defined on entity classes. Validated at startup, cached, and reusable. Replaced by @Query in Spring Data JPA.

Q22. What is optimistic vs pessimistic locking? Optimistic: uses @Version column, detects conflicts at commit time. Pessimistic: locks database rows using SELECT FOR UPDATE, prevents concurrent access.

Q23. What is the @Version annotation? Implements optimistic locking. Hibernate checks the version column on UPDATE; if it changed since read, throws OptimisticLockException.

Q24. What is a Hibernate Dialect? Translates HQL/JPQL to database-specific SQL. Different databases have different SQL syntax (LIMIT vs ROWNUM, sequences vs auto-increment).

Q25. What is `spring.jpa.hibernate.ddl-auto`? Controls schema generation: `none` (no action), `validate` (check schema), `update` (alter tables), `create` (drop and create), `create-drop` (create on startup, drop on shutdown). Production should use `validate` or `none`.

Spring Data JPA Questions (Q26-Q50)

Q26. What is Spring Data JPA? An abstraction layer on top of JPA that generates repository implementations from interfaces at runtime, eliminating boilerplate data access code.

Q27. What is the difference between CrudRepository and JpaRepository? JpaRepository extends CrudRepository and adds JPA-specific methods: `flush()`, `saveAndFlush()`, `deleteAllInBatch()`, and supports pagination and sorting.

Q28. How does query method derivation work? Spring Data parses method names and generates JPQL. `findByEmailAndStatus(String email, Status status)` becomes `SELECT u FROM User u WHERE u.email = ?1 AND u.status = ?2`.

Q29. What is the @Query annotation? Defines custom JPQL or native SQL queries on repository methods when method name derivation is insufficient.

Q30. What is @Modifying? Required on @Query methods that modify data (UPDATE, DELETE). Must be used with @Transactional.

Q31. How does pagination work? Pass `Pageable` parameter to repository method. Returns `Page<T>` with content, total count, page info. `PageRequest.of(page, size, sort)` creates Pageable.

Q32. What is a Specification? A reusable query predicate. Implements `Specification<T>` interface. Composable with `.and()`, `.or()`. Requires `JpaSpecificationExecutor`.

Q33. What is an @EntityGraph? Defines which associations to fetch eagerly for a specific query, overriding default fetch types. Prevents N+1 without JOIN FETCH in query string.

Q34. What is a DTO projection? Selecting only specific columns into a DTO instead of full entity. Reduces data transfer and avoids loading unnecessary relationships.

Q35. What is the difference between interface and class projections? Interface projections: Spring generates proxy implementing getter methods. Class projections: constructor-based, JPQL `SELECT NEW`.

Q36. How to handle soft deletes? Use `@SQLDelete` and `@Where` annotations or manually add `deletedAt` column with global filter.

Q37. What is Auditing in Spring Data JPA? Auto-populating `createdBy`, `createdDate`, `lastModifiedBy`, `lastModifiedDate` using `@EnableJpaAuditing` and `@CreatedDate`, `@LastModifiedDate` annotations.

Q38. What is @Transactional(readOnly = true)? Tells Hibernate to skip dirty checking for entities loaded in this transaction, improving performance. May also route to read replicas.

Q39. What happens if @Transactional is on a private method? It does NOT work. Spring AOP creates proxies for public methods only. @Transactional on private methods is silently ignored.

Q40. What is the Open Session in View pattern? Keeps the Hibernate Session open until the view is rendered (controller returns response). Prevents LazyInitializationException but can cause N+1 queries in views. `spring.jpa.open-in-view=false` to disable (recommended).

Q41. How to implement custom repository methods? Create a custom interface + implementation class. Spring Data merges it with the auto-generated implementation.

Q42. What is QueryByExample? Creates queries from entity instances. `productRepository.findAll(Example.of(probe))` where probe is a partially filled entity.

Q43. How to use multiple data sources? Configure separate DataSource, EntityManagerFactory, and TransactionManager beans. Use `@EnableJpaRepositories(basePackages, entityManagerFactoryRef)` per datasource.

Q44. What is @Param in @Query? Binds method parameters to named query parameters: `@Query("... WHERE u.email = :email") User findByEmail(@Param("email") String email)`.

Q45. What is saveAndFlush() vs save()? `save()` persists but may delay SQL until transaction commit. `saveAndFlush()` immediately executes SQL, useful when you need the generated ID right away.

Q46. How to call stored procedures? Use `@Procedure` annotation or `@NamedStoredProcedureQuery` on the entity.

Q47. What is @Lock in Spring Data JPA? Applies pessimistic or optimistic lock mode to a query: `@Lock(LockModeType.PESSIMISTIC_WRITE)`.

Q48. How to handle database migrations? Use Flyway or Liquibase. Never use `ddl-auto: update` in production. Version all schema changes as migration scripts.

Q49. What is the difference between findById and getById? `findById()` returns `Optional<T>`, immediately executes SELECT. `getReferenceById()` returns a lazy proxy, throws EntityNotFoundException on access if not found.

Q50. How to test repositories? Use `@DataJpaTest` with embedded H2. It auto-configures an in-memory database, scans for entities and repositories, wraps tests in transactions that rollback.

Performance and Architecture Questions (Q51-Q75)

Q51. How to solve the N+1 problem? Four solutions: JOIN FETCH in JPQL, @EntityGraph, @BatchSize on collections, DTO projections that avoid relationships entirely.

Q52. How to optimize batch inserts? Use SEQUENCE strategy (not IDENTITY), set `hibernate.jdbc.batch_size=50`, enable `order_inserts=true`, flush and clear EntityManager periodically.

Q53. What is connection pool sizing best practice? Formula: connections = (core_count * 2) + disk_spindles. For most apps, 10-20 connections with HikariCP.

Q54. How to detect slow queries? Enable `hibernate.generate_statistics=true` and `show_sql=true`. Use slow query logging in the database. Monitor with APM tools.

Q55. What is the difference between JPQL JOIN and JOIN FETCH? JOIN filters results but doesn't load the association. JOIN FETCH loads the association into the entity (initializes the lazy proxy).

Q56. When to use native queries? Database-specific functions, window functions, CTEs, full-text search, performance-critical queries, complex aggregations.

Q57. How to handle large result sets? Use pagination (Pageable), streaming (Stream), or ScrollableResults. Never load millions of entities into memory.

Q58. What is the impact of CascadeType.ALL? Cascades all operations including REMOVE. Can cause unexpected deletions. Use specific cascades (PERSIST, MERGE) unless parent truly owns child lifecycle.

Q59. How to prevent LazyInitializationException? Fetch data within @Transactional, use JOIN FETCH, use @EntityGraph, map to DTO inside service layer, or disable Open Session in View.

Q60. What is database indexing strategy for JPA? Index columns used in WHERE, JOIN, and ORDER BY clauses. Use `@Index` on `@Table` or manage via Flyway/Liquibase migrations.

Q61. How to implement multi-tenancy? Database per tenant (separate datasources), Schema per tenant (schema switching), or Discriminator column (shared table with tenant_id filter).

Q62. What is OSIV (Open Session in View) and why disable it? Keeps Session open through controller response rendering. Causes lazy queries in view layer (N+1 risk). Disable with `spring.jpa.open-in-view=false`.

Q63. How to handle concurrent updates? Use @Version for optimistic locking. Handle OptimisticLockException with retry logic in service layer.

Q64. What is the difference between first and second level cache? L1: per Session, automatic, cannot be disabled. L2: per SessionFactory, shared, configurable, requires explicit opt-in per entity.

Q65. How to monitor Hibernate in production? Enable statistics, export metrics to Prometheus/Grafana (via Micrometer), log slow queries, monitor connection pool metrics (HikariCP).

Q66. What is @DynamicUpdate? Generates UPDATE SQL with only changed columns, not all columns. Useful for tables with many columns.

Q67. What is @NaturalId? Marks a business key (like email, SSN). Hibernate provides a cache-friendly lookup by natural ID.

Q68. How to handle inheritance in JPA? Three strategies: SINGLE_TABLE (one table, discriminator column), JOINED (table per class with JOIN), TABLE_PER_CLASS (separate tables, no JOIN).

Q69. When to use SINGLE_TABLE vs JOINED inheritance? SINGLE_TABLE: best performance (no JOINs), nullable columns. JOINED: normalized, no nulls, but slower due to JOINs.

Q70. What is @MappedSuperclass vs @Inheritance? @MappedSuperclass: shares fields but no polymorphic queries. @Inheritance: full polymorphism with `FROM BaseEntity` queries.

Q71. How to implement soft delete with JPA? Add `deletedAt` column, override delete with `@SQLDelete(sql = "UPDATE ... SET deleted_at = NOW()")`, filter with `@Where(clause = "deleted_at IS NULL")`.

Q72. What is EntityManager.flush()? Synchronizes persistence context with database (executes pending SQL) WITHOUT committing the transaction. Useful before native queries.

Q73. What is the difference between em.persist() and repo.save()? `persist()` is JPA standard, only works on new entities. `save()` calls `persist()` for new or `merge()` for existing (based on ID presence).

Q74. How to handle time zones in JPA? Store as `Instant` or `OffsetDateTime` in UTC. Configure JVM timezone: `-Duser.timezone=UTC`. Use `@Column(columnDefinition =`

`"TIMESTAMP WITH TIME ZONE")`.

Q75. What is the best entity equals/hashCode strategy? Use business key (email, ISBN), never the generated ID (changes from null to value on persist). If no business key, use UUID set in constructor.

Architect-Level Questions (Q76-Q100)

Q76. Design a multi-tenant SaaS persistence layer. Use discriminator-based tenancy with `tenant_id` column. Implement Hibernate filter to automatically add `WHERE tenant_id = ?`. Use `ThreadLocal` to store current tenant.

Q77. How to handle database migrations in CI/CD? Flyway migration scripts versioned in Git. CI pipeline runs `flyway:migrate` before deployment. Rollback scripts for each migration. Never use `ddl-auto`.

Q78. Design read/write splitting with JPA. Configure two datasources (primary for writes, replica for reads). Use `@Transactional(readOnly = true)` to route to replica. Implement `AbstractRoutingDataSource`.

Q79. How to implement CQRS with JPA? Write model: full entities with JPA for commands. Read model: DTO projections or separate read-optimized views. Optionally synchronize via events.

Q80. Design an audit logging system with JPA. Use Hibernate Envers or custom `@EntityListeners` with `@PrePersist`, `@PreUpdate`, `@PreRemove`. Store audit trail in separate audit tables.

Q81. How to handle schema evolution in microservices? Each service owns its schema. Use backward-compatible migrations (add columns, never remove). Blue-green deployment with migration windows.

Q82. Design a high-performance bulk import system. Stateless Session, JDBC batch inserts, disable auto-commit, clear persistence context every N rows, use database COPY command for massive imports.

Q83. How to implement event sourcing with JPA? Store events as entities in an events table. Rebuild state by replaying events. Use `@DomainEvents` in Spring Data for publishing.

Q84. How to handle distributed caching with Hibernate L2? Use Hazelcast or Redis as L2 cache provider. Configure cache regions per entity. Handle cache invalidation across cluster nodes.

Q85. Design a full-text search with JPA. Use Hibernate Search (Lucene/Elasticsearch integration) or native database full-text indexes with native queries.

Q86. How to profile and optimize a slow JPA application? Enable Hibernate statistics, identify N+1 queries, check for missing indexes, analyze query plans (EXPLAIN), add DTO projections, configure batch

fetching.

Q87. Design a versioned API with different entity shapes. Use DTO projection per API version. Keep entity stable; vary DTOs. Version endpoints: `/api/v1/users`, `/api/v2/users`.

Q88. How to handle eventual consistency between services? Outbox pattern: write event to outbox table in same transaction, background process publishes to Kafka. Consumer is idempotent.

Q89. When would you NOT use an ORM? Extreme performance requirements, complex reporting (use raw SQL/views), simple CRUD with few entities (use Spring JDBC), NoSQL databases.

Q90. Design a data archival strategy with JPA. Partition tables by date. Batch job moves old data to archive tables. Use separate entity mappings for archive. Maintain foreign key integrity.

Q91. How to handle database connection issues gracefully? HikariCP connection validation, circuit breaker pattern (Resilience4j), retry with exponential backoff, health checks for database connectivity.

Q92. Design the persistence layer for an e-commerce platform. User, Product, Order, OrderItem, Payment, Address entities. Aggregate root pattern (Order is aggregate root for OrderItems). Optimistic locking on inventory. Event-driven stock updates.

Q93. How to implement row-level security with JPA? Hibernate `@Filter` with `@FilterDef`. Enable filter per Session with tenant/user context. Or use database-level row-level security (PostgreSQL RLS).

Q94. What is the Outbox Pattern? Write domain event to an outbox table within the same transaction as the business operation. A separate process reads the outbox and publishes to message broker. Guarantees at-least-once delivery.

Q95. How to handle large BLOB/CLOB data? Use `@Lob` with lazy loading. Store large files in object storage (S3), save only reference in DB. Use streaming for large reads.

Q96. Design a polymorphic search across entity types. Use SINGLE_TABLE inheritance with discriminator. Query base entity with Specifications. Or use Elasticsearch for cross-entity search.

Q97. How to implement database sharding with JPA? Custom `AbstractRoutingDataSource` that routes based on shard key. Each shard has own EntityManagerFactory. Application manages shard selection.

Q98. What is the Unit of Work pattern in Hibernate? The Session IS the Unit of Work. It tracks all entity changes (inserts, updates, deletes) and flushes them as a batch in a single transaction on commit.

Q99. How to implement the Specification pattern for complex filters? Create reusable Specification classes. Compose with `.and()`, `.or()`. Support dynamic filter UI. Test each specification independently.

Q100. Design a reporting system that doesn't impact OLTP performance. Use read replica for reports. Materialize views for complex aggregations. Use Spring Batch for scheduled report generation. DTO projections for lightweight queries.

Q101. What is the difference between Spring Data JPA and Spring Data JDBC? JPA: full ORM, lazy loading, caching, entity lifecycle. JDBC: no ORM, no lazy loading, no session cache. JDBC is simpler but less powerful.

Q102. How to handle database transactions across multiple repositories? One @Transactional on the service method wraps all repository calls in a single transaction. Don't put @Transactional on repositories.

Q103. What is the N+1 problem in aggregations? Loading aggregate data by iterating entities instead of using GROUP BY queries. Solution: use @Query with aggregation functions or native SQL.

Part 25: Coding Exercises (20)

Exercise 1: Basic Entity with Auditing

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class Article {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(columnDefinition = "TEXT")
    private String content;

    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy
    @Column(updatable = false)
    private String createdBy;
}
```

Exercise 2: Composite Key Entity

```
@Embeddable
public class EnrollmentId implements Serializable {
    private Long studentId;
    private Long courseId;
    // equals() and hashCode()
}

@Entity
public class Enrollment {
```

```
    @EmbeddedId
    private EnrollmentId id;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("studentId")
    private Student student;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("courseId")
    private Course course;

    private LocalDate enrolledDate;
    private String grade;
}
```

Exercise 3: OneToMany with Cascade and OrphanRemoval

```
@Entity
public class ShoppingCart {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy = "cart", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<CartItem> items = new ArrayList<>();

    public void addItem(Product product, int quantity) {
        CartItem item = new CartItem(this, product, quantity);
        items.add(item);
    }

    public void removeItem(CartItem item) {
        items.remove(item);
        item.setCart(null);
    }
}
```

Exercise 4: Inheritance Mapping (SINGLE_TABLE)

```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "payment_type")
public abstract class Payment {
    @Id @GeneratedValue private Long id;
    private BigDecimal amount;
    private LocalDateTime paidAt;
}

@Entity
@DiscriminatorValue("CREDIT_CARD")
public class CreditCardPayment extends Payment {
    private String cardNumber;
    private String expiryDate;
}

@Entity
@DiscriminatorValue("BANK_TRANSFER")
public class BankTransferPayment extends Payment {
    private String bankCode;
    private String accountNumber;
}
```

Exercise 5: Query Method Derivation

```java
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByDepartmentAndStatusOrderBySalaryDesc(
        String department, EmployeeStatus status);

    Optional<Employee> findFirstByDepartmentOrderBySalaryDesc(
        String department); // Highest paid in department

    long countByDepartment(String department);
    boolean existsByEmail(String email);
    List<Employee> findTop5ByOrderBySalaryDesc(); // Top 5 earners
}
```

Exercise 6: Custom @Query with DTO Projection

```java
public record DepartmentStats(
    String department, long employeeCount, BigDecimal avgSalary) {}

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    @Query("SELECT new com.app.dto.DepartmentStats(" +
           "e.department, COUNT(e), AVG(e.salary)) " +
           "FROM Employee e GROUP BY e.department " +
           "ORDER BY AVG(e.salary) DESC")
    List<DepartmentStats> getDepartmentStatistics();
}
```

Exercise 7: Pagination with Specification

```java
@Service
public class EmployeeSearchService {

    @Autowired private EmployeeRepository repo;

    public Page<Employee> search(String name, String dept,
                                 BigDecimal minSalary, Pageable pageable) {
        Specification<Employee> spec = Specification.where(null);
        if (name != null) spec = spec.and(nameContains(name));
        if (dept != null) spec = spec.and(inDepartment(dept));
        if (minSalary != null) spec = spec.and(salaryGreaterThan(minSalary));
        return repo.findAll(spec, pageable);
    }

    private Specification<Employee> nameContains(String name) {
        return (r, q, cb) -> cb.like(cb.lower(r.get("name")),
            "%" + name.toLowerCase() + "%");
    }

    private Specification<Employee> inDepartment(String dept) {
        return (r, q, cb) -> cb.equal(r.get("department"), dept);
    }

    private Specification<Employee> salaryGreaterThan(BigDecimal min) {
        return (r, q, cb) -> cb.greaterThanOrEqualTo(r.get("salary"), min);
    }
}
```

Exercise 8: Optimistic Locking

```java
@Entity
public class InventoryItem {
    @Id private Long id;
    private String productName;
    private int quantity;

    @Version
    private Long version; // Optimistic lock column

    public void decrementStock(int amount) {
        if (quantity < amount)
            throw new InsufficientStockException();
        this.quantity -= amount;
    }
}

@Service
public class InventoryService {
    @Transactional
    @Retryable(value = OptimisticLockException.class, maxAttempts = 3)
    public void reserveStock(Long itemId, int amount) {
        InventoryItem item = repo.findById(itemId).orElseThrow();
        item.decrementStock(amount);
        // If another thread updated concurrently, version mismatch
        // throws OptimisticLockException -> retry
    }
}
```

Exercise 9: Entity Lifecycle Callbacks

```java
@Entity
public class Order {
    @Id @GeneratedValue private Long id;
    private BigDecimal amount;
    private String status;

    @PrePersist  void prePersist()  { log.info("About to INSERT order"); }
    @PostPersist void postPersist() { log.info("INSERTed order id={}", id); }
    @PreUpdate   void preUpdate()   { log.info("About to UPDATE order {}", id);
```

```
}
    @PostUpdate  void postUpdate()  { log.info("UPDATEd order {}", id); }
    @PreRemove   void preRemove()   { log.info("About to DELETE order {}", id);
}
    @PostRemove  void postRemove()  { log.info("DELETEd order {}", id); }
    @PostLoad    void postLoad()    { log.info("LOADed order {}", id); }
}
```

Exercise 10: Bulk Update with @Modifying

```
public interface UserRepository extends JpaRepository<User, Long> {
    @Modifying(clearAutomatically = true)
    @Transactional
    @Query("UPDATE User u SET u.status = 'INACTIVE' " +
           "WHERE u.lastLoginAt < :cutoff")
    int deactivateInactiveUsers(@Param("cutoff") LocalDateTime cutoff);
}
```

Exercise 11: ManyToMany with Extra Columns

```
// When join table needs extra columns, use an entity instead of @ManyToMany
@Entity
public class ProjectAssignment {
    @EmbeddedId
    private ProjectAssignmentId id;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("employeeId")
    private Employee employee;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("projectId")
    private Project project;

    private String role;          // Extra column
    private LocalDate assignedDate; // Extra column
}
```

Exercise 12: Custom Repository Implementation

```java
public interface UserRepositoryCustom {
    List<User> findUsersWithComplexCriteria(UserSearchCriteria criteria);
}

@Repository
public class UserRepositoryCustomImpl implements UserRepositoryCustom {
    @PersistenceContext
    private EntityManager em;

    @Override
    public List<User> findUsersWithComplexCriteria(UserSearchCriteria c) {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<User> query = cb.createQuery(User.class);
        Root<User> root = query.from(User.class);

        List<Predicate> predicates = new ArrayList<>();
        if (c.getName() != null)
            predicates.add(cb.like(root.get("name"), "%" + c.getName() + "%"));
        if (c.getMinAge() != null)
            predicates.add(cb.ge(root.get("age"), c.getMinAge()));

        query.where(predicates.toArray(new Predicate[0]));
        return em.createQuery(query).getResultList();
    }
}

// Combine with Spring Data
public interface UserRepository extends JpaRepository<User, Long>,
                                        UserRepositoryCustom { }
```

Exercise 13: Interface-Based Projection

```java
public interface UserSummary {
    Long getId();
    String getFullName();
    String getEmail();

    @Value("#{target.firstName + ' ' + target.lastName}")
    String getDisplayName(); // Computed from entity fields
```

```
}

public interface UserRepository extends JpaRepository<User, Long> {
    List<UserSummary> findByStatus(UserStatus status);
}
```

Exercise 14: Soft Delete

```
@Entity
@SQLDelete(sql = "UPDATE products SET deleted_at = NOW() WHERE id = ?")
@Where(clause = "deleted_at IS NULL")
public class Product {
    @Id @GeneratedValue private Long id;
    private String name;
    private LocalDateTime deletedAt;
}
// productRepository.deleteById(1L);
// Executes: UPDATE products SET deleted_at = NOW() WHERE id = 1
// productRepository.findAll();
// Executes: SELECT * FROM products WHERE deleted_at IS NULL
```

Exercise 15: EntityGraph for Selective Fetching

```
@Entity
@NamedEntityGraph(name = "Customer.withOrdersAndItems",
    attributeNodes = {
        @NamedAttributeNode(value = "orders",
            subgraph = "orders-items")
    },
    subgraphs = {
        @NamedSubgraph(name = "orders-items",
            attributeNodes = @NamedAttributeNode("items"))
    })
public class Customer { ... }

public interface CustomerRepository extends JpaRepository<Customer, Long> {
    @EntityGraph("Customer.withOrdersAndItems")
    Optional<Customer> findById(Long id);
}
```

Exercise 16: Stored Procedure Call

```
public interface ReportRepository extends JpaRepository<Report, Long> {

    @Procedure(name = "generate_monthly_report")
    void generateMonthlyReport(@Param("month") int month,
                               @Param("year") int year);
}
```

Exercise 17: Multiple DataSources

```
@Configuration
@EnableJpaRepositories(
    basePackages = "com.app.primary.repository",
    entityManagerFactoryRef = "primaryEntityManagerFactory",
    transactionManagerRef = "primaryTransactionManager")
public class PrimaryDataSourceConfig {

    @Primary
    @Bean
    @ConfigurationProperties("spring.datasource.primary")
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create().build();
    }
}
```

Exercise 18: Query by Example

```
@Transactional(readOnly = true)
public List<Product> searchByExample(String name, String category) {
    Product probe = new Product();
    probe.setName(name);
    probe.setCategory(category);

    ExampleMatcher matcher = ExampleMatcher.matching()
        .withIgnoreCase()
        .withStringMatcher(ExampleMatcher.StringMatcher.CONTAINING)
        .withIgnoreNullValues();
```

```
    return productRepository.findAll(Example.of(probe, matcher));
}
```

Exercise 19: Batch Processing with JPA

```
@Transactional
public void batchInsert(List<Employee> employees) {
    for (int i = 0; i < employees.size(); i++) {
        entityManager.persist(employees.get(i));
        if (i % 50 == 0) { // Flush every 50
            entityManager.flush();
            entityManager.clear(); // Free memory
        }
    }
}
```

Exercise 20: Testing Repository with @DataJpaTest

```
@DataJpaTest
class UserRepositoryTest {

    @Autowired private UserRepository userRepository;
    @Autowired private TestEntityManager entityManager;

    @Test
    void shouldFindByEmail() {
        User user = new User("John", "john@test.com");
        entityManager.persistAndFlush(user);

        Optional<User> found = userRepository.findByEmail("john@test.com");

        assertThat(found).isPresent();
        assertThat(found.get().getName()).isEqualTo("John");
    }

    @Test
    void shouldReturnEmptyForNonExistentEmail() {
        Optional<User> found = userRepository.findByEmail("x@test.com");
        assertThat(found).isEmpty();
```

```
    }

    @Test
    void shouldPaginateResults() {
        for (int i = 0; i < 25; i++) {
            entityManager.persist(new User("User" + i, "u" + i + "@test.com"));
        }
        entityManager.flush();

        Page<User> page = userRepository.findAll(PageRequest.of(0, 10));

        assertThat(page.getContent()).hasSize(10);
        assertThat(page.getTotalElements()).isEqualTo(25);
        assertThat(page.getTotalPages()).isEqualTo(3);
    }
}
```

Scenario-Based Debugging Questions (S1-S10)

S1. Your app generates 500 SQL queries for a single API call. How do you diagnose and fix this? Diagnosis: Enable `hibernate.generate_statistics=true`. The log shows query count per session. Root cause is almost always N+1. Fix: identify which lazy collection is triggering queries, add JOIN FETCH or @EntityGraph for that specific endpoint.

S2. Users report that updates are "not saving" intermittently. What could cause this? Possible causes: (1) Entity is detached (accessed outside @Transactional boundary), (2) Self-invocation -- @Transactional method called from same class, (3) merge() used but original reference modified instead of returned reference, (4) @Transactional(readOnly=true) accidentally applied to a write method.

S3. Your batch job processes 100K records and runs out of memory. How to fix? Process in pages (500 at a time), call `entityManager.flush()` and `entityManager.clear()` after each page. Consider using StatelessSession for pure bulk operations. Monitor heap usage with `-Xmx` and GC logs.

S4. After adding a new enum value, production data starts showing wrong statuses. What happened? The entity uses `@Enumerated(EnumType.ORDINAL)`. Adding or reordering enum values shifts ordinal positions, corrupting all existing data. Fix: migrate to `EnumType.STRING` and run a data migration script.

S5. Your application takes 45 seconds to start. What Hibernate-related issues could cause this? Possible causes: (1) Multiple SessionFactory instances being created, (2) `ddl-auto: update`

scanning and altering large schemas, (3) Thousands of @NamedQuery validations, (4) Entity scanning across too many packages. Check: duplicate @EntityScan, multiple persistence units.

S6. Connection pool exhaustion occurs under load. What do you investigate? Check: (1) Long-running transactions (external API calls inside @Transactional), (2) Missing @Transactional causing separate connection per repository call, (3) Leak detection not configured, (4) Pool size too small for concurrent request count. Enable `hikari.leak-detection-threshold=30000`.

S7. Two users buying the last item both succeed, and inventory goes negative. How to prevent this? Add `@Version` field to Inventory entity (optimistic locking). The second user's UPDATE fails because version changed. Wrap in `@Retryable` with `OptimisticLockException`. For critical inventory, consider pessimistic locking: `@Lock(LockModeType.PESSIMISTIC_WRITE)`.

S8. A developer reports that deleteAll() on a table with 1 million rows takes 10 minutes. How to optimize? `deleteAll()` loads ALL entities first, then deletes one by one (1M SELECT + 1M DELETE). Use `deleteAllInBatch()` which executes a single `DELETE FROM table` SQL. For conditional deletes, use `@Modifying @Query("DELETE FROM Entity WHERE ...")` bulk operation.

S9. Customer reports seeing another customer's data. How could Hibernate cause this? A shared Session across threads (non-thread-safe). Or Open Session in View with thread pool recycling. Or second-level cache returning stale/incorrect data after a cache invalidation failure. Check thread safety of EntityManager injection and OSIV settings.

S10. After migrating from MySQL to PostgreSQL, some queries fail with syntax errors. What went wrong? Native queries use MySQL-specific syntax (LIMIT without OFFSET, backtick identifiers, GROUP_CONCAT). JPQL queries are portable, native queries are NOT. Identify all `nativeQuery = true` methods and verify PostgreSQL compatibility. Also check Dialect configuration in application.yml.

End of Guide

Document Version: 2.0 (Enhanced Edition) **Last Updated:** March 2026 **Topics Covered:** 25 Parts, 103+ Interview Questions, 10 Scenario-Based Debugging Questions, 20 Coding Exercises